Data Loading & Checks

In [ ]:
```
import pandas as pd
import numpy as np
from mlxtend.frequent_patterns import fpgrowth, association_rules
from datetime import datetime
import time
import matplotlib.pyplot as plt
from statsmodels.tsa.stattools import adfuller
import seaborn as sns
```

In [60]:
```
df = pd.read_csv('Groceries data train.csv')
df['Date'] = pd.to_datetime(df['Date'], format='%d/%m/%Y')
```

In [61]:
```
print(df.head())
```

```
   User_id       Date itemDescription    year  month  day  day_of_week
0   2351.0 2014-01-01         cleaner  2014.0    1.0  1.0          2.0
1   2226.0 2014-01-01         sausage  2014.0    1.0  1.0          2.0
2   1922.0 2014-01-01  tropical fruit  2014.0    1.0  1.0          2.0
3   2943.0 2014-01-01      whole milk  2014.0    1.0  1.0          2.0
4   1249.0 2014-01-01    citrus fruit  2014.0    1.0  1.0          2.0
```

In [62]:
```
print("Missing values")
df.isnull().sum()
```

```
Missing values
```

Out[62]:
```
User_id            7603
Date               7603
itemDescription    7603
year               7603
month              7603
day                7603
day_of_week        7603
dtype: int64
```

In [63]:
```
missing_rows = df[df.isnull().any(axis=1)]
missing_rows
```

Out[63]:

	User_id	Date	itemDescription	year	month	day	day_of_week
19382	NaN	NaT	NaN	NaN	NaN	NaN	NaN
19383	NaN	NaT	NaN	NaN	NaN	NaN	NaN
19384	NaN	NaT	NaN	NaN	NaN	NaN	NaN
19385	NaN	NaT	NaN	NaN	NaN	NaN	NaN
19386	NaN	NaT	NaN	NaN	NaN	NaN	NaN
...	...	...	...	...	...	...	...
26980	NaN	NaT	NaN	NaN	NaN	NaN	NaN
26981	NaN	NaT	NaN	NaN	NaN	NaN	NaN
26982	NaN	NaT	NaN	NaN	NaN	NaN	NaN
26983	NaN	NaT	NaN	NaN	NaN	NaN	NaN
26984	NaN	NaT	NaN	NaN	NaN	NaN	NaN

7603 rows × 7 columns

In [64]:
```
# train data shape
df.shape
```

Out[64]: (26985, 7)

In [65]:
```
# drop empty rows
train_data = df.dropna(how='all')
train_data.shape
```

Out[65]: (19382, 7)

Observation

- The columns in the dataset are, `User_id` , `Date` `itemDescription` , `year` , `month` , `day` , `day_of_week`
- The data contains empty rows which needs to be removed before any further analysis.

Exploratory Data Analysis

In [66]:
```
# Uniqueness
print(f"No. of unique users: {train_data['User_id'].nunique()}")
print(f"No. of unique items: {train_data['itemDescription'].nunique()}")
```

```
No. of unique users: 3493
No. of unique items: 167
```

In [67]:
```
train_data['transaction_id'] = train_data['User_id'].astype(str) + '_' + train_data['Date
```

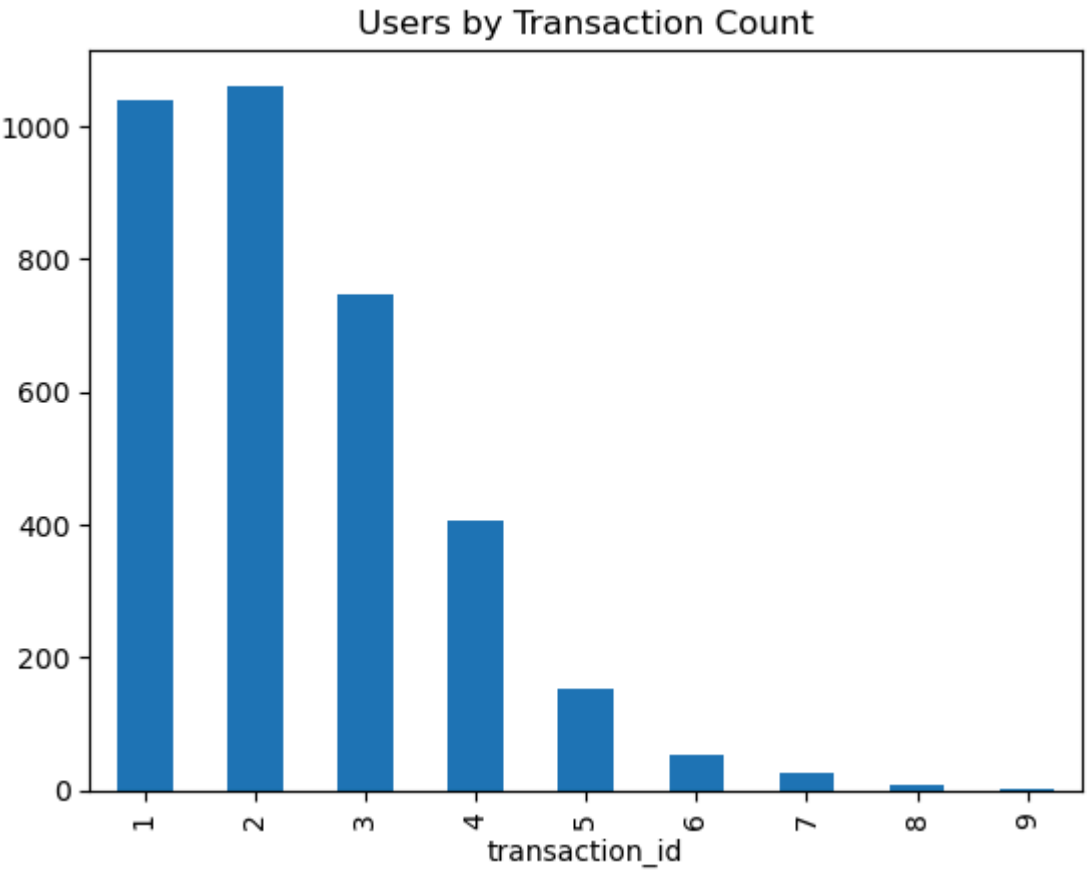
```
:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_gu
ide/indexing.html#returning-a-view-versus-a-copy
  train_data['transaction_id'] = train_data['User_id'].astype(str) + '_' + train_data['Dat
e'].dt.strftime('%Y-%m-%d')
```

In [68]:
```
txn_per_user = train_data.groupby('User_id')['transaction_id'].nunique()
txn_count = txn_per_user.value_counts().sort_index()
txn_count
```

Out[68]:
```
transaction_id
1    1039
2    1061
3     747
4     406
5     152
6      52
7      27
8       7
9       2
Name: count, dtype: int64
```

In [69]:
```
txn_count[txn_count.index <= 10].plot(kind='bar', title='Users by Transaction Count')
plt.show()
```



Observations

1. Basic Stats:
   - unique users: 3,493
   - unique items: 167
2. No. of Transactions by Users(Fig: Users by Transaction Count):
   - This distribution tells that there are many users who just made only one or two or just three transactions in the entire period of this dataset.
   - This is very useful data for the overall recommendation project because it suggests that we do not have much historical data on users.

In [70]:
```
basket_sizes = train_data.groupby('transaction_id')['itemDescription'].nunique()
basket_sizes
```

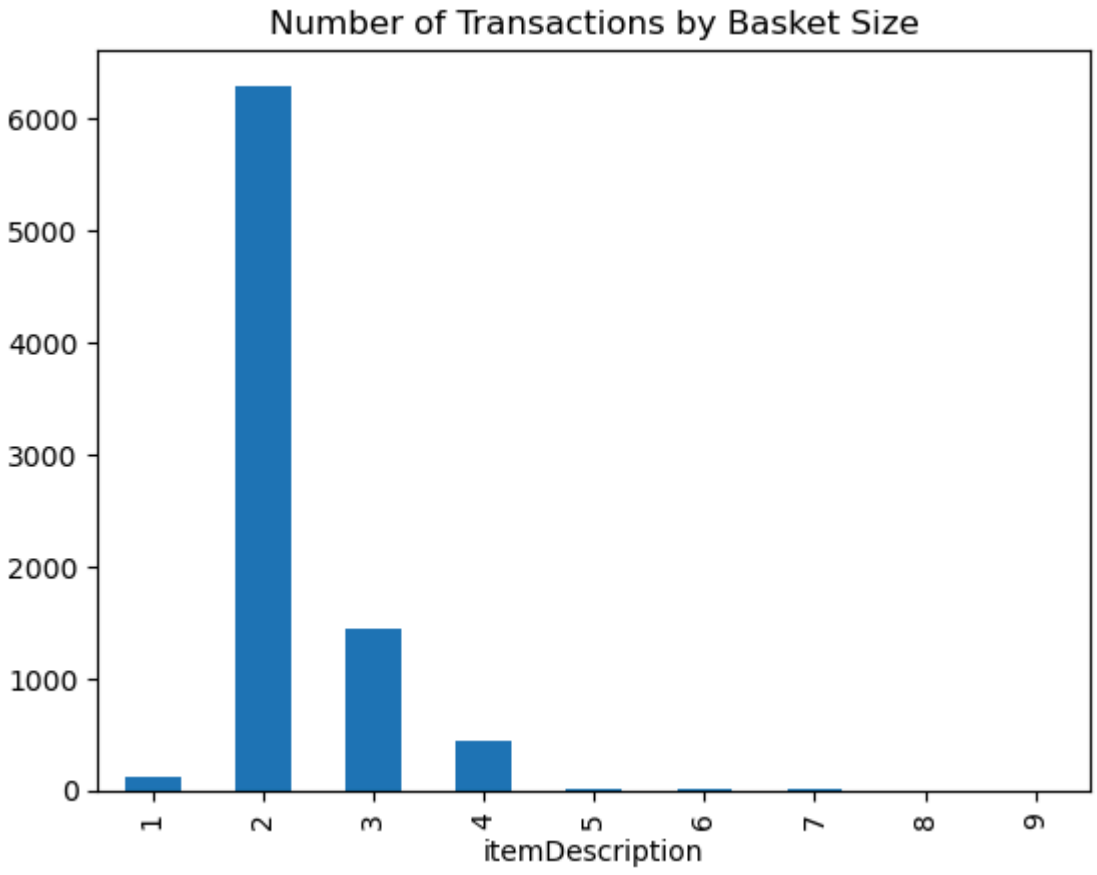
Out[70]:
```
transaction_id
1000.0_2014-06-24    3
1001.0_2014-07-02    3
1001.0_2014-12-12    2
1001.0_2015-01-20    2
1002.0_2014-04-26    2
                    ..
4997.0_2014-05-01    2
4999.0_2014-05-19    2
4999.0_2014-09-04    2
5000.0_2014-09-03    2
5000.0_2014-11-16    2
Name: itemDescription, Length: 8361, dtype: int64
```

In [71]:
```
basket_size_counts = basket_sizes.value_counts().sort_index()
basket_size_counts
```

Out[71]:
```
itemDescription
1     131
2    6297
3    1443
4     446
5      20
6      10
7       7
8       6
9       1
Name: count, dtype: int64
```

In [72]:
```
basket_size_counts.plot(kind='bar', title='Number of Transactions by Basket Size')
```

Out[72]:
```
<Axes: title={'center': 'Number of Transactions by Basket Size'}, xlabel='itemDescriptio
n'>
```



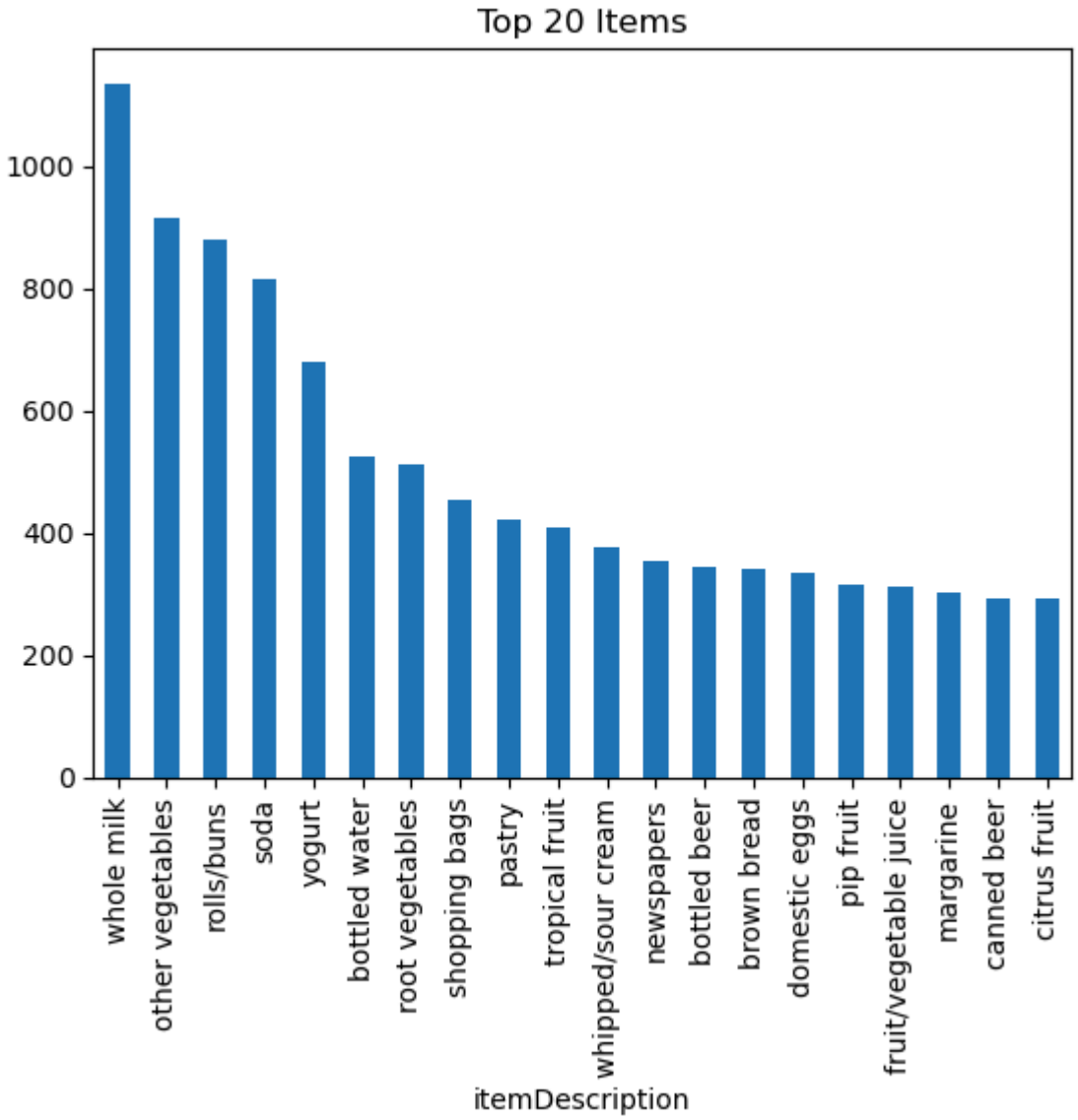
Observation: Basket Size

- Most of the transactions have two items in their basket. The total number is 6297.
- There is also a significant number(1443) in which there are three items in the basket.

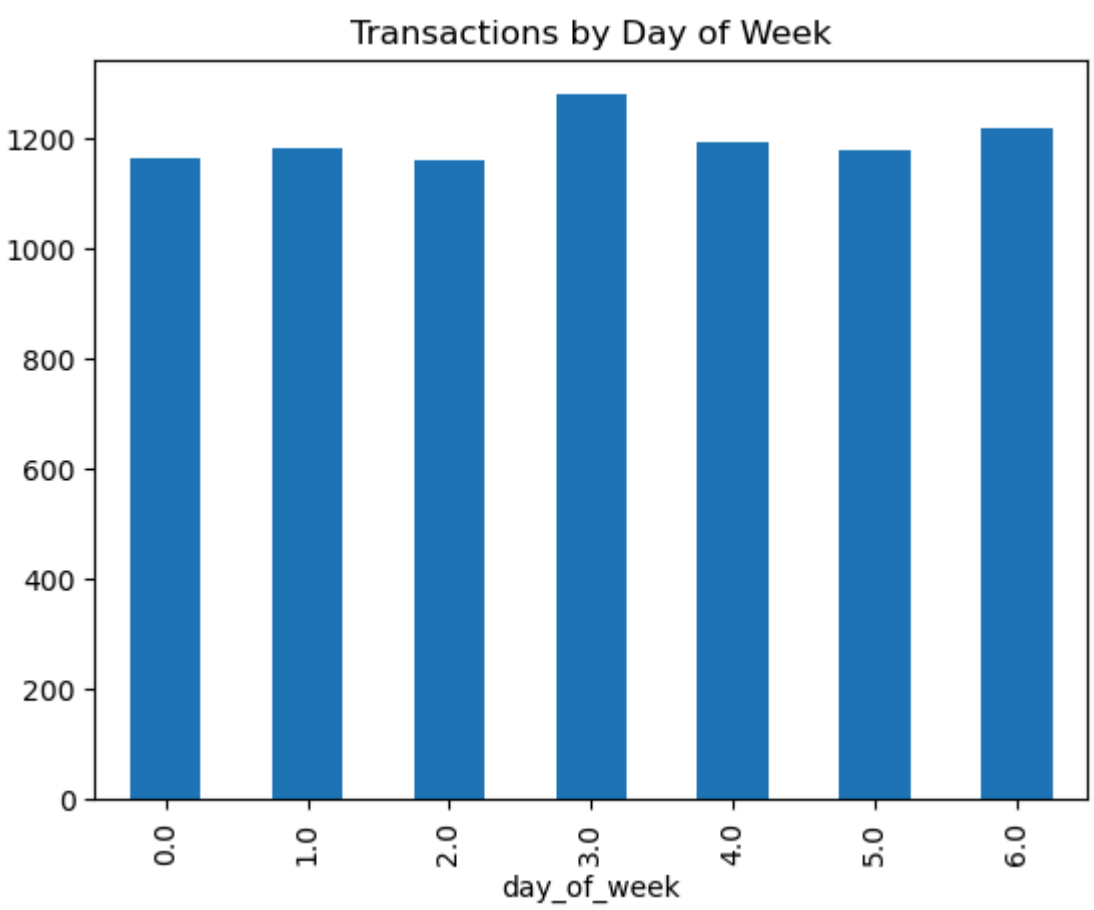
- After 4 times the numbers are very insignificant.

This suggests focusing on a pattern containing 2 or 3 items.

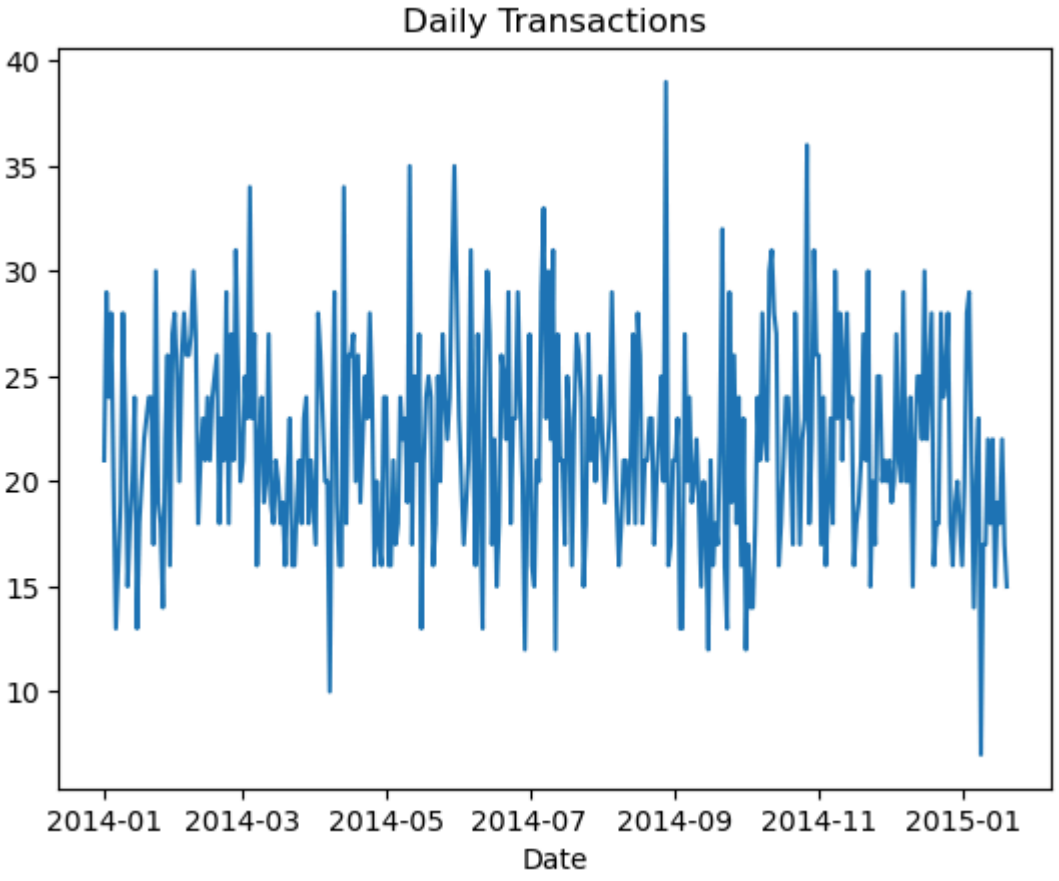
In [74]:
```
train_data['itemDescription'].value_counts().head(20).plot(kind='bar', title='Top 20 Item
plt.show()
```



In [73]:
```
train_data.groupby('day_of_week')['transaction_id'].nunique().plot(kind='bar', title='Tra
plt.show()
```



In [75]:
```
train_data.groupby(train_data['Date'].dt.date)['transaction_id'].nunique().plot(title='Da
plt.show()
```



In [76]:
```
month_counts = train_data.groupby('month').size().rename('count').reset_index()
print(month_counts)
```

```
    month  count
0     1.0   2609
1     2.0   1547
2     3.0   1491
3     4.0   1506
4     5.0   1625
5     6.0   1525
6     7.0   1623
7     8.0   1535
8     9.0   1350
9    10.0   1555
10   11.0   1496
11   12.0   1520
```

In [77]:
```
dow_month = train_data.groupby(['day_of_week', 'month']).size().unstack(fill_value=0)
print("Day of week by month transaction counts:")
print(dow_month)
```

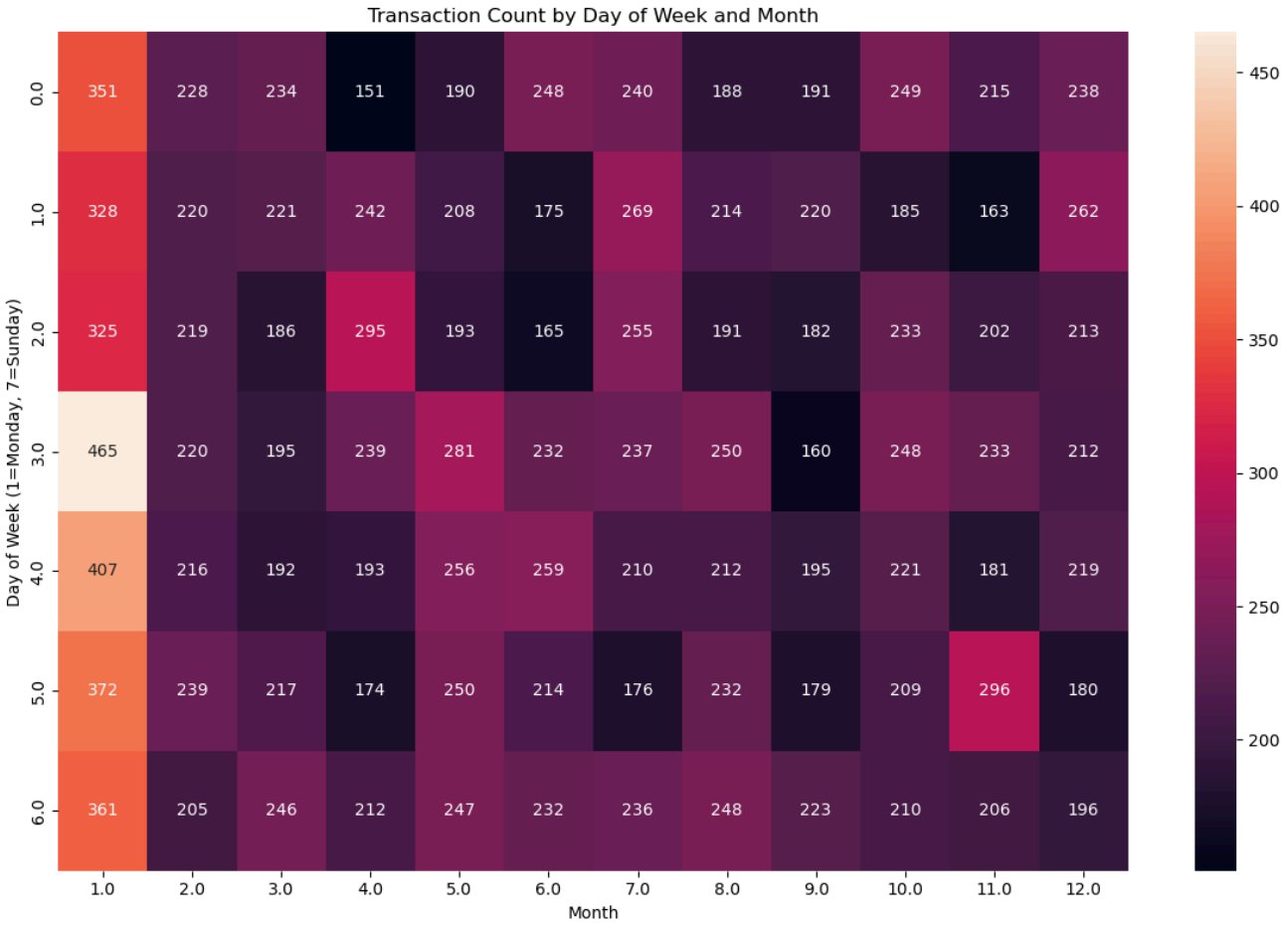
```
Day of week by month transaction counts:
month         1.0   2.0   3.0   4.0   5.0   6.0   7.0   8.0   9.0   10.0  11.0  \
day_of_week
0.0            351   228   234   151   190   248   240   188   191   249   215
1.0            328   220   221   242   208   175   269   214   220   185   163
2.0            325   219   186   295   193   165   255   191   182   233   202
3.0            465   220   195   239   281   232   237   250   160   248   233
4.0            407   216   192   193   256   259   210   212   195   221   181
5.0            372   239   217   174   250   214   176   232   179   209   296
6.0            361   205   246   212   247   232   236   248   223   210   206

month         12.0
day_of_week
0.0            238
1.0            262
2.0            213
3.0            212
4.0            219
5.0            180
6.0            196
```

Observations

1. Top 20 items
    - Whole milk is the most frequent item bought.
    - It is then followed by other significant frequent items "other vegetables," "rolls/buns," and "soda.
2. Transactions by Day of Week
    - This data shows the transaction or shopping activity is consistent each day.
    - It just shows a slight increase in the transaction on Wednesday.
3. Daily Transaction
    - The daily transaction numbers fluctuate between 15-35 transactions per day.
    - This pattern of fluctuation does not show any trend or seasonality in the dataset.

In [78]:
```
plt.figure(figsize=(12, 8))
sns.heatmap(dow_month, annot=True, fmt='d')
plt.title('Transaction Count by Day of Week and Month')
plt.xlabel('Month')
plt.ylabel('Day of Week (1=Monday, 7=Sunday)')
plt.tight_layout()
plt.show()
```



Observation

1. This heatmap shows the transaction counts on the day in a particular month. T
2. This analysis is helpful in finding the activeness of customers on a particular day in a particular month. This is helpful in finding seasonal patterns.
3. From the heatmap we can see that, in January month we can observe that Wednesday and Thursday have significantly higher transactions.

In [79]:
```
top_items = train_data['itemDescription'].value_counts().nlargest(10).index.tolist()
top_trends = train_data[train_data['itemDescription'].isin(top_items)].groupby(['month',
print("Monthly trends for top 10 items:")
top_items
```

```
Monthly trends for top 10 items:
```

Out[79]:
```
['whole milk',
 'other vegetables',
 'rolls/buns',
 'soda',
 'yogurt',
 'bottled water',
 'root vegetables',
 'shopping bags',
 'pastry',
 'tropical fruit']
```

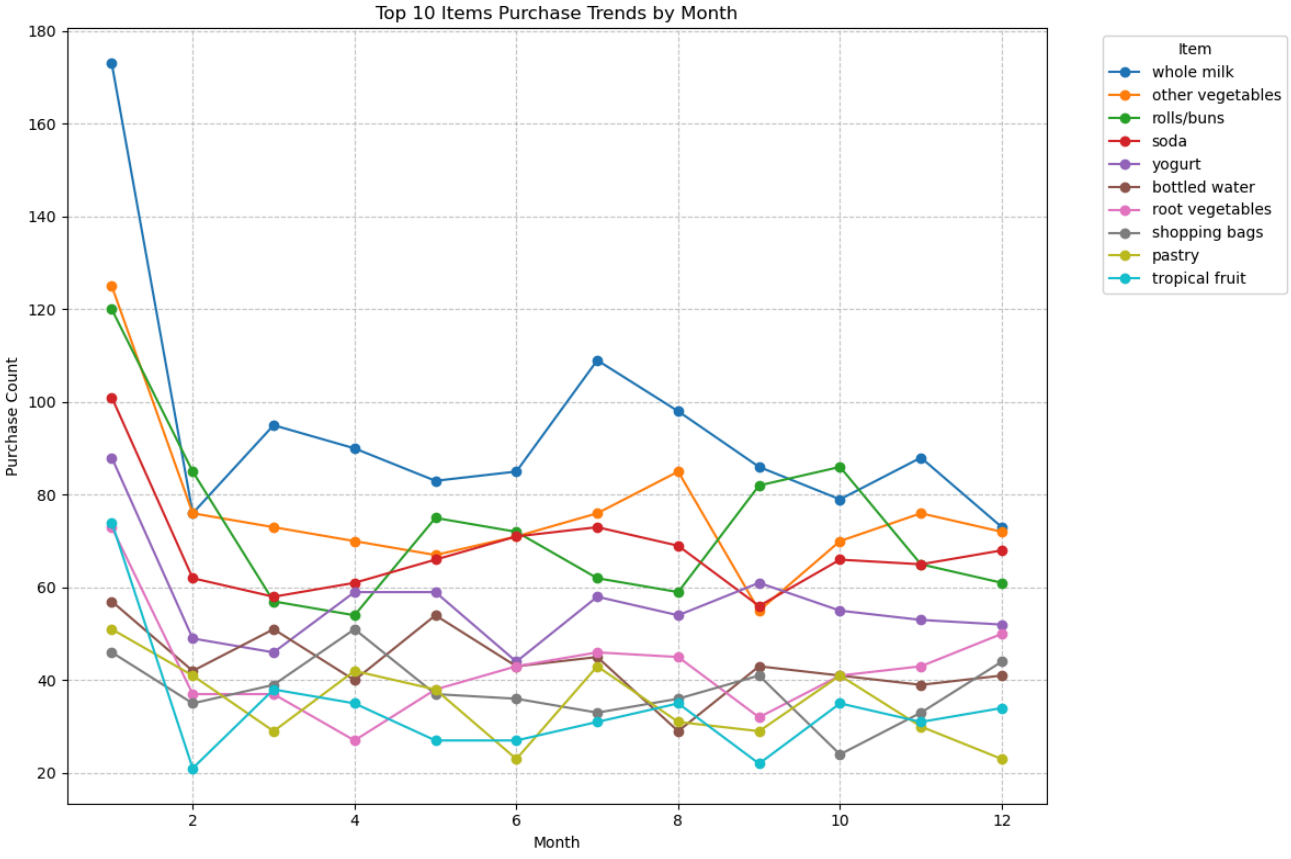
In [80]:
```
top_trends
```

Out[80]:

itemDescription	bottled water	other vegetables	pastry	rolls/buns	root vegetables	shopping bags	soda	tropical fruit	w
month									
1.0	57	125	51	120	73	46	101	74	
2.0	42	76	41	85	37	35	62	21	
3.0	51	73	29	57	37	39	58	38	
4.0	40	70	42	54	27	51	61	35	
5.0	54	67	38	75	38	37	66	27	
6.0	43	71	23	72	43	36	71	27	
7.0	45	76	43	62	46	33	73	31	
8.0	29	85	31	59	45	36	69	35	
9.0	43	55	29	82	32	41	56	22	
10.0	41	70	41	86	41	24	66	35	
11.0	39	76	30	65	43	33	65	31	
12.0	41	72	23	61	50	44	68	34	

In [81]:

```
plt.figure(figsize=(12, 8))
for item in top_items:
    plt.plot(top_trends.index, top_trends[item], label=item, marker='o')
plt.title('Top 10 Items Purchase Trends by Month')
plt.xlabel('Month')
plt.ylabel('Purchase Count')
plt.legend(title='Item', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.grid(True, linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```



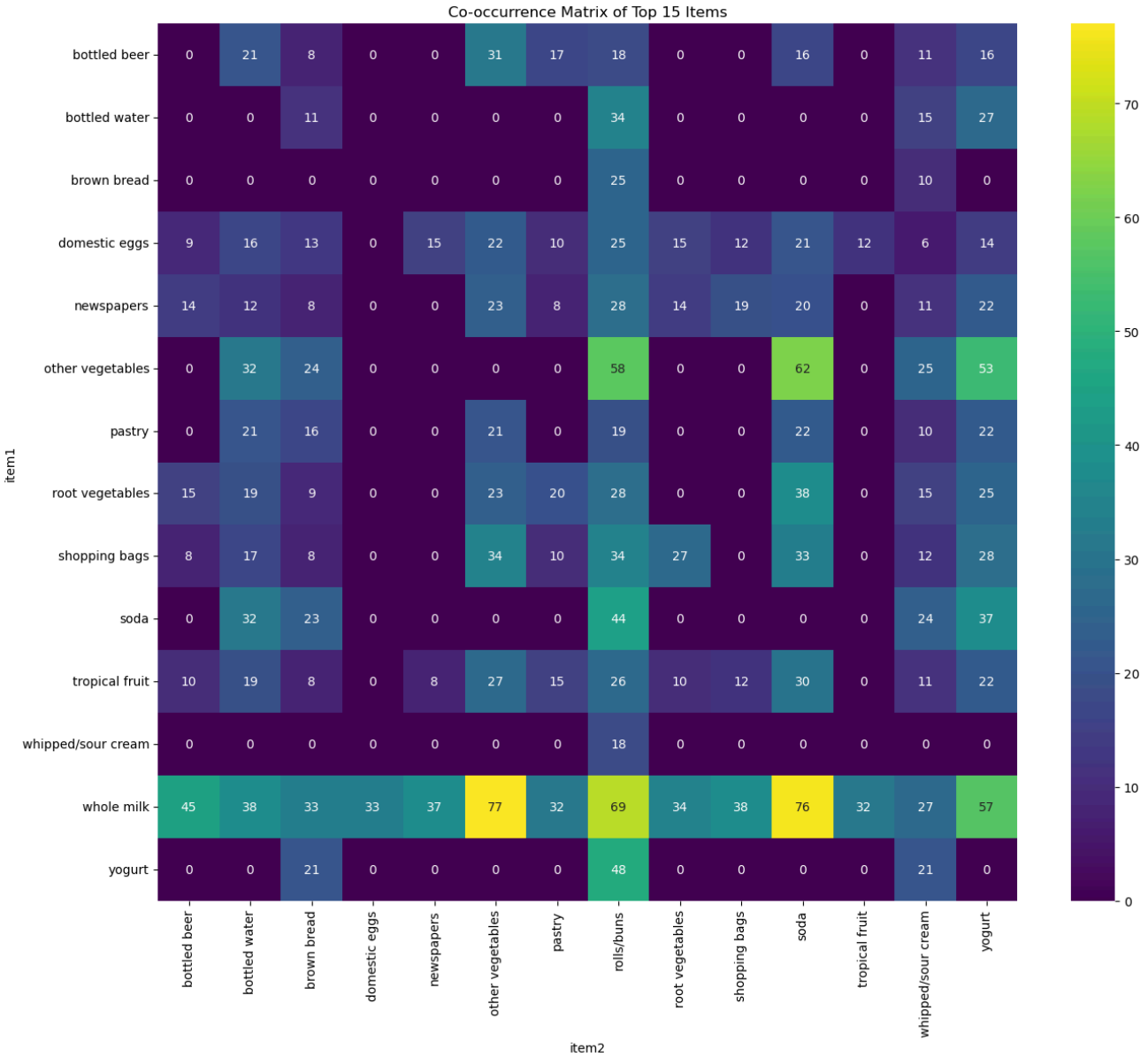
In [82]:

```
top_items = train_data['itemDescription'].value_counts().nlargest(15).index.tolist()
item_data = train_data[train_data['itemDescription'].isin(top_items)]
transactions = item_data.groupby(['User_id', 'Date'])['itemDescription'].apply(list).rese

# Create co-occurrence matrix
from itertools import combinations
co_occur = {}
for _, row in transactions.iterrows():
    items = row['itemDescription']
    for item1, item2 in combinations(set(items), 2):
        if (item1, item2) in co_occur:
            co_occur[(item1, item2)] += 1
        elif (item2, item1) in co_occur:
            co_occur[(item2, item1)] += 1
        else:
            co_occur[(item1, item2)] = 1

# Convert to DataFrame for visualization
co_occur_df = pd.DataFrame([(i, j, v) for (i, j), v in co_occur.items()],
                           columns=['item1', 'item2', 'count'])
co_matrix = pd.pivot_table(co_occur_df, values='count',
                           index='item1', columns='item2', fill_value=0)

plt.figure(figsize=(14, 12))
sns.heatmap(co_matrix, annot=True, cmap='viridis', fmt='g')
plt.title('Co-occurrence Matrix of Top 15 Items')
plt.tight_layout()
plt.show()
```



In [83]:
```
#  top 3 items by purchase frequency
top_3_items = train_data['itemDescription'].value_counts().nlargest(3).index.tolist()
print("Top 3 items:", top_3_items)

# group by week
weekly_item_counts = train_data[train_data['itemDescription'].isin(top_3_items)]\
    .groupby([pd.Grouper(key='Date', freq='W'), 'itemDescription'])\
    .size().unstack(fill_value=0)

# check stationarity
for item in top_3_items:
    print(f"\nStationarity Analysis for: {item}")

    # Get item data
    ts = weekly_item_counts[item]

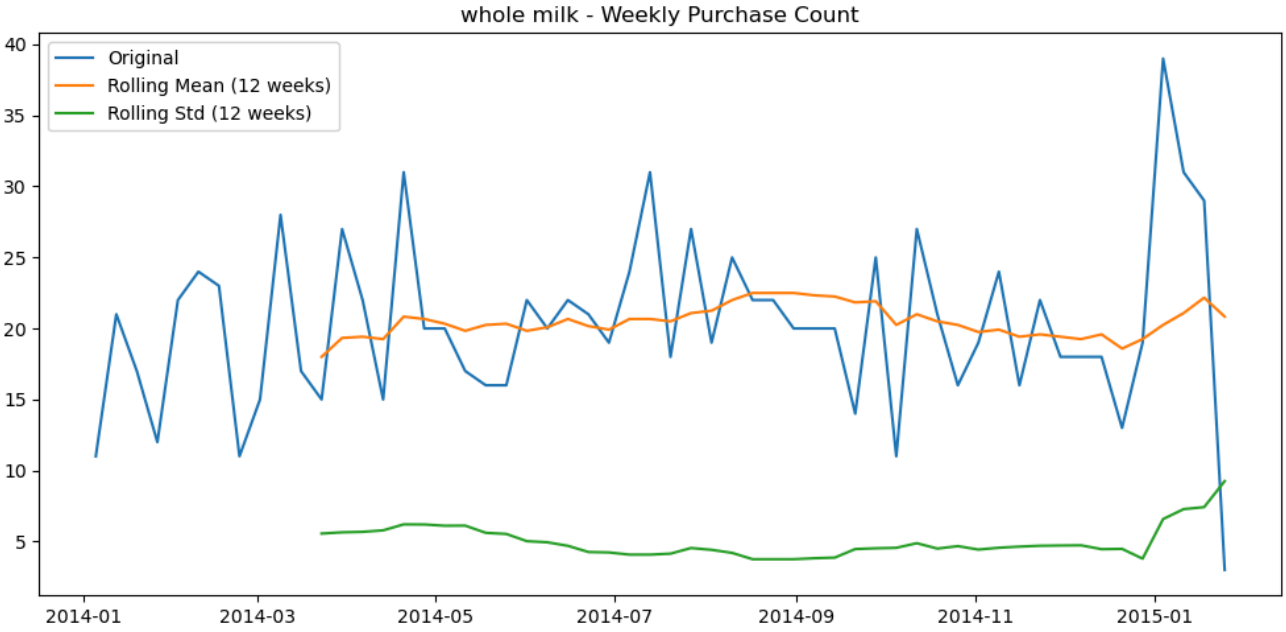
    # calculate 12-week rolling statistics
    rolling_mean = ts.rolling(window=12).mean()
    rolling_std = ts.rolling(window=12).std()

    plt.figure(figsize=(10, 5))
    plt.plot(ts, label='Original')
    plt.plot(rolling_mean, label='Rolling Mean (12 weeks)')
    plt.plot(rolling_std, label='Rolling Std (12 weeks)')
    plt.legend()
    plt.title(f'{item} - Weekly Purchase Count')
    plt.tight_layout()
    plt.show()

    # Basic ADF test(Bhatt, 2025)
    result = adfuller(ts.dropna())
    print(f'ADF Statistic: {result[0]:.4f}')
    print(f'p-value: {result[1]:.4f}')
    print('Stationary? {"Yes" if p value <= 0.05 else "No"}')
```

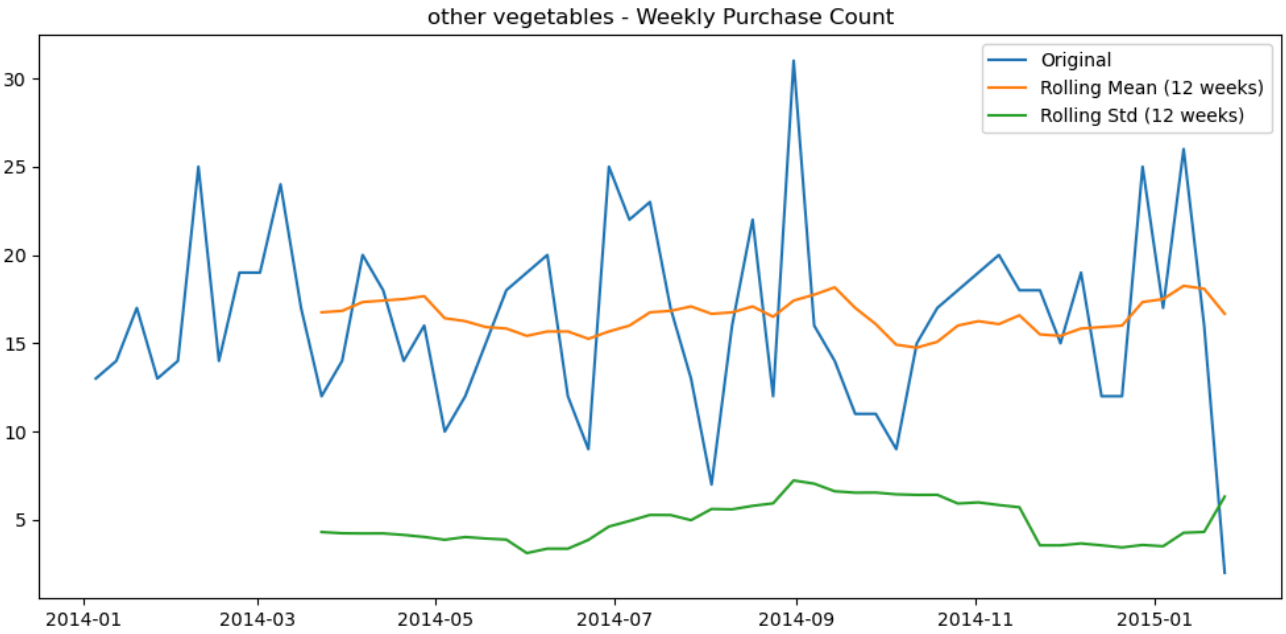
```
Top 3 items: ['whole milk', 'other vegetables', 'rolls/buns']

Stationarity Analysis for: whole milk
```



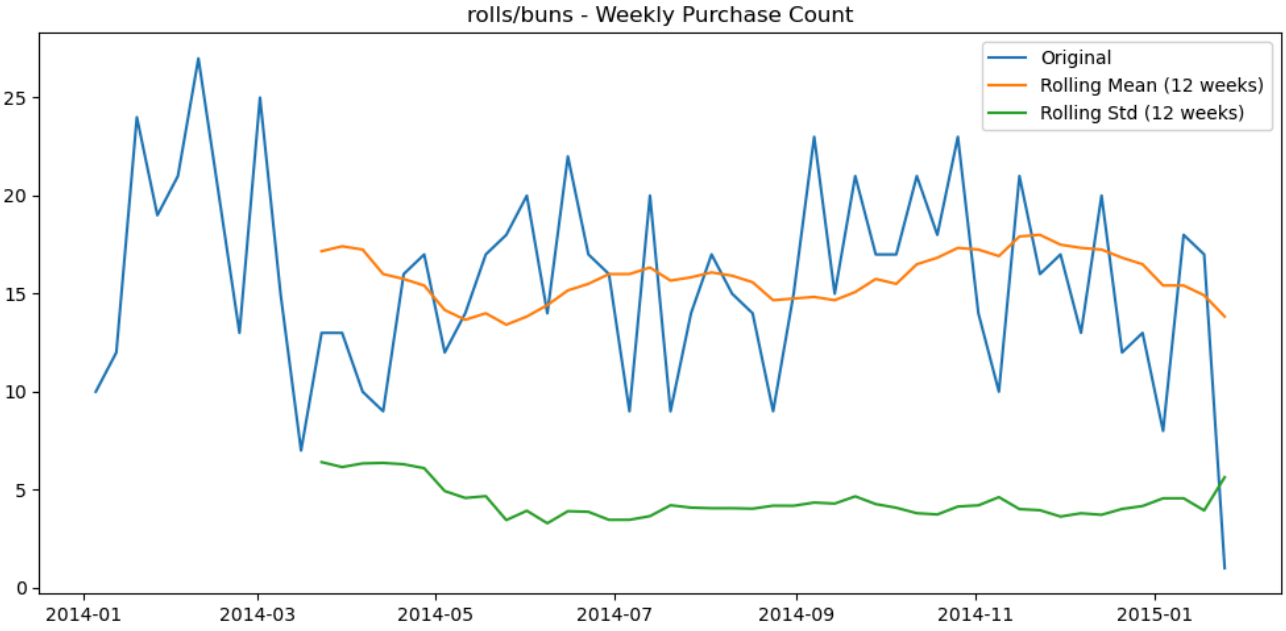

```
ADF Statistic: -7.2125
p-value: 0.0000
Stationary? {"Yes" if p value <= 0.05 else "No"}

Stationarity Analysis for: other vegetables
```




```
ADF Statistic: -5.6813
p-value: 0.0000
Stationary? {"Yes" if p value <= 0.05 else "No"}

Stationarity Analysis for: rolls/buns
```




```
ADF Statistic: -3.6019
p-value: 0.0057
Stationary? {"Yes" if p value <= 0.05 else "No"}
```

Observations

1. Top 10 items Purchased by month
    - Some seasonality can be seen in the transaction of whole milk. It peaks and July with some fluctuation in between.
    - The graph shows that all products have a sharp drop in the Feb month.
2. Cooccurance Heatmap
    - This is a very important visualization as it shows which two popular items are frequently bought together.
    - We can see that whole milk co-occurs very frequently with other-vegetables.
    - This is a very sparse matrix and shows that most of the items rarely co-occur or do not co-occur at all.
    - Whole milk co-occurs all top 15 items.
    - This helps in visualizing the association rules from our pattern mining results.

3. Stationarity Time Series Analysis
    - The chart shows weekly patterns of popular items.
    - All the p values from the ADF test are less than 0.05, suggesting that the transaction data with time is stationary.
    - All three graphs show shows stable underlying pattern with slight variations. This shows consistent behavior in purchases.

Pattern Mining

Transaction Preparation

In [ ]:
```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
def prepare_transactions(data, user_id=None):
    # filter by user_id
    if user_id is not None:
        data = data[data['User_id'] == user_id]
        if data.empty:
            return None
    # group items by user and date
    transactions = data.groupby(['User_id', 'Date'])['itemDescription'].apply(list).reset
    # list of item lists
    item_lists = transactions['itemDescription'].tolist()
    # encode item lists
    te = TransactionEncoder()
    te_ary = te.fit(item_lists).transform(item_lists)
    encoded_df = pd.DataFrame(te_ary, columns=te.columns_)
    encoded_df.insert(0, 'Date', transactions['Date'])
    encoded_df.insert(0, 'User_id', transactions['User_id'])
    encoded_df.set_index(['User_id', 'Date'], inplace=True)
    return encoded_df
```

Explanation

- This function is required to convert the data into a suitable form so that the mining algorithm implemented later can use them.
- It also takes user_id as a parameter and returns encoded transactions specific to that user.

In [85]:
```
sample_data = train_data.head(100)
sample_transactions = prepare_transactions(sample_data)
sample_transactions.head()
```

Out[85]:

User_id	Date	Instant food products	beef	berries	bottled beer	bottled water	brown bread	butter	butter milk	candles	candy
1066.0	2014-01-02	False	False	False	True	False	False	False	False	False	False
1248.0	2014-01-02	False	False	False	False	False	False	True	False	False	False
1249.0	2014-01-01	False	False	False	False	False	False	False	False	False	False
1265.0	2014-01-02	False	False	False	False	False	False	False	False	False	False
1365.0	2014-01-02	False	False	False	False	False	False	False	False	False	False

5 rows × 54 columns

FP-GROWTH

In [ ]:

```python
# Function to mine patterns using FP-Growth #inspired from(Aditya, 2023)
def mine_patterns(data, min_support=0.01, max_length=5, min_confidence=0.3, user_id=None)
    start_time = time.time()
    # Prepare transactions
    transactions = prepare_transactions(data, user_id)
    if transactions is None or transactions.empty:
        print("No transactions found.")
        return pd.DataFrame(), pd.DataFrame(), time.time() - start_time
    # Mine frequent itemsets using FP-Growth
    try:
        # Convert to boolean
        items_df = transactions.astype(bool)
        # Apply FP-Growth algorithm
        frequent_itemsets = fpgrowth(
            items_df,
            min_support=min_support,
            max_len=max_length,
            use_colnames=True
        )
        # Generate association rules
        if not frequent_itemsets.empty:
            rules = association_rules(
                frequent_itemsets,
                metric='confidence',
                min_threshold=min_confidence
            )
        else:
            rules = pd.DataFrame()
    except Exception as e:
        print(f"Error mining patterns: {e}")
        frequent_itemsets = pd.DataFrame()
        rules = pd.DataFrame()

    execution_time = time.time() - start_time
    return frequent_itemsets, rules, execution_time
```

Explanation

- We have chosen the FP-Growth algorithm over Apriori because it is more efficient and faster for big datasets does not generate all possible candidate items and uses an FP-tree to find frequent items(HackerTraining, 2025).
- This function calls the prepare_transactions function internally.
- This function takes data, support, confidence, user_id as input and returns frequent_itemsets, rules, execution_time.

In [88]:
```
frequent_itemsets, rules, _ = mine_patterns(train_data, min_support=0.001, min_confidence
```

In [89]:
```
rules
```

Out[89]:

	antecedents	consequents	antecedent support	consequent support	support	confidence	lift	repres
0	(salty snack)	(rolls/buns)	0.019615	0.101902	0.002033	0.103659	1.017241	
1	(sausage)	(whole milk)	0.025715	0.130487	0.002751	0.106977	0.819828	
2	(frankfurter)	(whole milk)	0.012199	0.130487	0.002033	0.166667	1.277269	
3	(frozen vegetables)	(whole milk)	0.027509	0.130487	0.002751	0.100000	0.766361	
4	(detergent)	(rolls/buns)	0.008492	0.101902	0.001196	0.140845	1.382166	
5	(pip fruit)	(rolls/buns)	0.037555	0.101902	0.003947	0.105096	1.031343	
6	(red/blush wine)	(soda)	0.011123	0.094606	0.001196	0.107527	1.136577	
7	(UHT-milk)	(other vegetables)	0.019854	0.106686	0.002153	0.108434	1.016384	
8	(candy)	(whole milk)	0.016027	0.130487	0.002153	0.134328	1.029440	
9	(napkins)	(whole milk)	0.024997	0.130487	0.002512	0.100478	0.770028	
10	(hamburger meat)	(rolls/buns)	0.017223	0.101902	0.001794	0.104167	1.022227	
11	(fruit/vegetable juice)	(rolls/buns)	0.036718	0.101902	0.003708	0.100977	0.990928	
12	(long life bakery product)	(whole milk)	0.020332	0.130487	0.002033	0.100000	0.766361	
13	(newspapers)	(whole milk)	0.041383	0.130487	0.004425	0.106936	0.819519	
14	(ham)	(whole milk)	0.010286	0.130487	0.001076	0.104651	0.802006	
15	(ice cream)	(whole milk)	0.012558	0.130487	0.001555	0.123810	0.948828	
16	(pot plants)	(yogurt)	0.008372	0.078579	0.001076	0.128571	1.636204	
17	(cat food)	(whole milk)	0.014233	0.130487	0.001794	0.126050	0.966001	
18	(chewing gum)	(other vegetables)	0.012199	0.106686	0.001316	0.107843	1.010848	
19	(waffles)	(whole milk)	0.020332	0.130487	0.002512	0.123529	0.946681	
20	(bottled beer)	(whole milk)	0.040545	0.130487	0.005382	0.132743	1.017294	
21	(processed cheese)	(rolls/buns)	0.011482	0.101902	0.001435	0.125000	1.226673	
22	(seasonal products)	(rolls/buns)	0.007774	0.101902	0.001076	0.138462	1.358776	

Recency calculation

In [90]:

```
def calculate_recency_weights(user_data):
    user_data = user_data.copy()
    user_data['DateConverted'] = pd.to_datetime(user_data['Date'])
    # most recent purchase
    max_date = user_data['DateConverted'].max()
    recency_weights = {}
```

```
    for item in user_data['itemDescription'].unique():
        # dates when this item was purchased
        item_dates = user_data[user_data['itemDescription'] == item]['DateConverted']
        if not item_dates.empty:
            #  most recent purchase date for this item
            most_recent = item_dates.max()
            # calc. days since most recent purchase
            days_since = (max_date - most_recent).days
            # recency weight with decay rate of 0.05
            recency_weights[item] = np.exp(-0.005 * days_since)  # Decay rate of 0.05
    return recency_weights
```

In [91]:
```
user_data = train_data[train_data['User_id'] == 3681]
user_data
```

Out[91]:

	User_id	Date	itemDescription	year	month	day	day_of_week	transaction_id
5	3681.0	2014-01-01	onions	2014.0	1.0	1.0	2.0	3681.0_2014-01-01
26	3681.0	2014-01-01	whipped/sour cream	2014.0	1.0	1.0	2.0	3681.0_2014-01-01
45	3681.0	2014-01-01	dishes	2014.0	1.0	1.0	2.0	3681.0_2014-01-01
6966	3681.0	2014-05-19	whipped/sour cream	2014.0	5.0	19.0	0.0	3681.0_2014-05-19
6989	3681.0	2014-05-19	specialty fat	2014.0	5.0	19.0	0.0	3681.0_2014-05-19
12583	3681.0	2014-09-06	margarine	2014.0	9.0	6.0	5.0	3681.0_2014-09-06
12602	3681.0	2014-09-06	frozen vegetables	2014.0	9.0	6.0	5.0	3681.0_2014-09-06

In [92]:
```
calculate_recency_weights(user_data)
```

Out[92]:
```
{'onions': 0.2893842179390506,
 'whipped/sour cream': 0.5769498103804866,
 'dishes': 0.2893842179390506,
 'specialty fat': 0.5769498103804866,
 'margarine': 1.0,
 'frozen vegetables': 1.0}
```

Explanation

This is one of the very important functions of this module.

- It finds the date of the most recent purchase history.
- For every product it counts the number of days from the most recent purchase history of the item.
- Now it applies the exponential decay function, the most recent purchase items get higher weightage and older purchases get less weightage.

The function is as follows:

- D_i = most recent purchase date of item i
- $D_{\max}$ = most recent purchase date in the dataset
- $\Delta_i = D_{\max} - D_i$ = no. of days since item i was last purchased
- $\lambda = 0.005$ = decay rate

The **recency weight** w_i for item i is calcualted as follows(Customers.ai, 2025):

$$w_i = e^{-0.005 \cdot (D_{\max} - D_i)}$$

- In the recency weight calculated above we can see "frozen vegetables", and "margarine" have a score of 1 because they were the most recent purchases, other items like "specialty fat" have lower scores.

In [93]:
```
frequent_itemsets, rules, _ = mine_patterns(train_data, min_support=0.001, min_confidence
rules
```

Out[93]:

	antecedents	consequents	antecedent support	consequent support	support	confidence	lift	representati
0	(onions)	(whipped/sour cream)	0.333333	0.666667	0.333333	1.0	1.5	
1	(whipped/sour cream)	(onions)	0.666667	0.333333	0.333333	0.5	1.5	
2	(onions)	(dishes)	0.333333	0.333333	0.333333	1.0	3.0	
3	(dishes)	(onions)	0.333333	0.333333	0.333333	1.0	3.0	
4	(dishes)	(whipped/sour cream)	0.333333	0.666667	0.333333	1.0	1.5	
5	(whipped/sour cream)	(dishes)	0.666667	0.333333	0.333333	0.5	1.5	
6	(onions, dishes)	(whipped/sour cream)	0.333333	0.666667	0.333333	1.0	1.5	
7	(onions, whipped/sour cream)	(dishes)	0.333333	0.333333	0.333333	1.0	3.0	
8	(dishes, whipped/sour cream)	(onions)	0.333333	0.333333	0.333333	1.0	3.0	
9	(onions)	(dishes, whipped/sour cream)	0.333333	0.333333	0.333333	1.0	3.0	
10	(dishes)	(onions, whipped/sour cream)	0.333333	0.333333	0.333333	1.0	3.0	
11	(whipped/sour cream)	(onions, dishes)	0.666667	0.333333	0.333333	0.5	1.5	
12	(whipped/sour cream)	(specialty fat)	0.666667	0.333333	0.333333	0.5	1.5	
13	(specialty fat)	(whipped/sour cream)	0.333333	0.666667	0.333333	1.0	1.5	
14	(frozen vegetables)	(margarine)	0.333333	0.333333	0.333333	1.0	3.0	
15	(margarine)	(frozen vegetables)	0.333333	0.333333	0.333333	1.0	3.0	

In [94]:
```
# function to calculate pattern recency
def calculate_pattern_recency(itemset, recency_weights):
    weights = [recency_weights.get(item, 0) for item in itemset]
    if not weights:
        return 0
    return sum(weights) / len(weights)
```

In [95]:
```
calculate_pattern_recency(['onions', 'dishes'], calculate_recency_weights(user_data))
```

Out[95]:
```
0.2893842179390506
```

Explanation: This function basically calculates the recency weight of an itemsets by taking mean of its individual recency weights.

Pattern Scoring Methods

Based on 'support', 'confidence', 'recency', 'combined',

In [96]:
```
def score_patterns(frequent_itemsets, rules=None, data=None, user_id=None, method='combir
    if frequent_itemsets.empty:
        return pd.DataFrame()
    scored_patterns = frequent_itemsets.copy()
    # scoring based on support
    if method == 'support':
        scored_patterns['score'] = scored_patterns['support']

    # if rules and want confidence based scoring
    elif method == 'confidence' and rules is not None and not rules.empty:
        itemset_scores = {}
        for _, rule in rules.iterrows():
            itemset = frozenset(rule['antecedents'].union(rule['consequents']))
            score = rule['confidence']
            if itemset in itemset_scores:
                itemset_scores[itemset] = max(itemset_scores[itemset], score)
            else:
                itemset_scores[itemset] = score
        scored_patterns['score'] = scored_patterns['itemsets'].apply(lambda x: itemset_sc
    # recency based scoring (transaction data and user_id)
    elif method == 'recency' and data is not None and user_id is not None:
        user_data = data[data['User_id'] == user_id]
        if user_data.empty: # support scoring if no user data
            scored_patterns['score'] = scored_patterns['support']
        else:
            recency_weights = calculate_recency_weights(user_data)
            scored_patterns['score'] = scored_patterns['itemsets'].apply(lambda itemset:
    #  using both support and recency
    elif method == 'combined' and data is not None and user_id is not None:
        user_data = data[data['User_id'] == user_id]
        if user_data.empty:
            scored_patterns['score'] = scored_patterns['support']
        else:
            recency_weights = calculate_recency_weights(user_data)
            scored_patterns['recency_score'] = scored_patterns['itemsets'].apply(lambda i
            max_support = scored_patterns['support'].max()
            if max_support > 0:
                scored_patterns['norm_support'] = scored_patterns['support'] / max_suppor
            else:
                scored_patterns['norm_support'] = scored_patterns['support']
            scored_patterns['score'] = 0.6 * scored_patterns['norm_support'] + 0.4 * scor
    #default to support
    else:
        scored_patterns['score'] = scored_patterns['support']
```

```
    return scored_patterns.sort_values('score', ascending=False)
```

Explanation

This function uses various scoring methods to score the patterns(Sharma, 2024). It takes frequent itemsets as input along with rules (optional), data and user_id (optional), method: "support", "confidence", "recency", or "combined" and returns a new data frame of frequent itemsets with a new score column.

The function behaves in the following manner on taking different inputs:

1. frequent_itemsets (required): It should be non-empty for any scoring.
2. rules (optional): The pattern will be matched with their best matching rule's confidence, rather than by support or recency.
3. data: the full transaction data needed for the calculation of recency.
4. user_id (optional): User whose history we want for calculating the recency.
5. method: one of the scoring methods:'support', 'confidence', 'recency', 'combined'
    - For the combined score, the final scoring is calculated as:
        - score = 0.6 × norm_support + 0.4 × recency_score.

Tests and Evaluation

In [191…
```
frequent_itemsets, rules, _ = mine_patterns(train_data, min_support=0.001, min_confidence
```

In [192…
```
frequent_itemsets
```

Out[192…

	support	itemsets
0	0.130487	(whole milk)
1	0.049635	(pastry)
2	0.019615	(salty snack)
3	0.101902	(rolls/buns)
4	0.025715	(sausage)
...	...	...
457	0.001316	(oil, soda)
458	0.001435	(processed cheese, rolls/buns)
459	0.001076	(whole milk, misc. beverages)
460	0.001076	(misc. beverages, rolls/buns)
461	0.001076	(seasonal products, rolls/buns)

462 rows × 2 columns

In [193…
```
rules
```

Out[193...

	antecedents	consequents	antecedent support	consequent support	support	confidence	lift	repres
0	(salty snack)	(rolls/buns)	0.019615	0.101902	0.002033	0.103659	1.017241	
1	(sausage)	(whole milk)	0.025715	0.130487	0.002751	0.106977	0.819828	
2	(frankfurter)	(whole milk)	0.012199	0.130487	0.002033	0.166667	1.277269	
3	(frozen vegetables)	(whole milk)	0.027509	0.130487	0.002751	0.100000	0.766361	
4	(detergent)	(rolls/buns)	0.008492	0.101902	0.001196	0.140845	1.382166	
5	(pip fruit)	(rolls/buns)	0.037555	0.101902	0.003947	0.105096	1.031343	
6	(red/blush wine)	(soda)	0.011123	0.094606	0.001196	0.107527	1.136577	
7	(UHT-milk)	(other vegetables)	0.019854	0.106686	0.002153	0.108434	1.016384	
8	(candy)	(whole milk)	0.016027	0.130487	0.002153	0.134328	1.029440	
9	(napkins)	(whole milk)	0.024997	0.130487	0.002512	0.100478	0.770028	
10	(hamburger meat)	(rolls/buns)	0.017223	0.101902	0.001794	0.104167	1.022227	
11	(fruit/vegetable juice)	(rolls/buns)	0.036718	0.101902	0.003708	0.100977	0.990928	
12	(long life bakery product)	(whole milk)	0.020332	0.130487	0.002033	0.100000	0.766361	
13	(newspapers)	(whole milk)	0.041383	0.130487	0.004425	0.106936	0.819519	
14	(ham)	(whole milk)	0.010286	0.130487	0.001076	0.104651	0.802006	
15	(ice cream)	(whole milk)	0.012558	0.130487	0.001555	0.123810	0.948828	
16	(pot plants)	(yogurt)	0.008372	0.078579	0.001076	0.128571	1.636204	
17	(cat food)	(whole milk)	0.014233	0.130487	0.001794	0.126050	0.966001	
18	(chewing gum)	(other vegetables)	0.012199	0.106686	0.001316	0.107843	1.010848	
19	(waffles)	(whole milk)	0.020332	0.130487	0.002512	0.123529	0.946681	
20	(bottled beer)	(whole milk)	0.040545	0.130487	0.005382	0.132743	1.017294	
21	(processed cheese)	(rolls/buns)	0.011482	0.101902	0.001435	0.125000	1.226673	
22	(seasonal products)	(rolls/buns)	0.007774	0.101902	0.001076	0.138462	1.358776	

In [196...

```
scored_patterns = score_patterns(frequent_itemsets, rules=rules, data=train_data, user_ic
scored_patterns
```

Out[196...

	support	itemsets	recency_score	norm_support	score
0	0.130487	(whole milk)	1.0	1.000000	1.000000
57	0.041383	(newspapers)	1.0	0.317140	0.590284
8	0.106686	(other vegetables)	0.0	0.817599	0.490559
3	0.101902	(rolls/buns)	0.0	0.780935	0.468561
5	0.094606	(soda)	0.0	0.725023	0.435014
...	...	...	...	...	...
139	0.001076	(pudding powder)	0.0	0.008249	0.004950
330	0.001076	(hard cheese, rolls/buns)	0.0	0.008249	0.004950
329	0.001076	(hard cheese, yogurt)	0.0	0.008249	0.004950
327	0.001076	(napkins, shopping bags)	0.0	0.008249	0.004950
461	0.001076	(seasonal products, rolls/buns)	0.0	0.008249	0.004950

462 rows × 5 columns

In [97]:
```
frequent_itemsets, rules, _ = mine_patterns(train_data, min_support=0.001, min_confidence
scored_patterns = score_patterns(frequent_itemsets, rules=rules, data=train_data, user_ic
scored_patterns.head(10)
```

Out[97]:

	support	itemsets	recency_score	norm_support	score
2	0.5	(whole milk)	1.000000	1.0	1.000000
3	0.5	(newspapers)	1.000000	1.0	1.000000
5	0.5	(whole milk, newspapers)	1.000000	1.0	1.000000
0	0.5	(hamburger meat)	0.245367	1.0	0.698147
1	0.5	(candles)	0.245367	1.0	0.698147
4	0.5	(hamburger meat, candles)	0.245367	1.0	0.698147

In [98]:
```
frequent_itemsets
```

Out[98]:

	support	itemsets
0	0.5	(hamburger meat)
1	0.5	(candles)
2	0.5	(whole milk)
3	0.5	(newspapers)
4	0.5	(hamburger meat, candles)
5	0.5	(whole milk, newspapers)

In [99]:
```
frequent_itemsets, rules, _ = mine_patterns(train_data, min_support=0.001, min_confidence
scored_patterns = score_patterns(frequent_itemsets, rules=rules, data=train_data, user_ic
scored_patterns.head(10)
```

Out[99]:

	support	itemsets	recency_score	norm_support	score
0	0.130487	(whole milk)	1.0	1.000000	1.000000
57	0.041383	(newspapers)	1.0	0.317140	0.590284
8	0.106686	(other vegetables)	0.0	0.817599	0.490559
3	0.101902	(rolls/buns)	0.0	0.780935	0.468561
5	0.094606	(soda)	0.0	0.725023	0.435014
387	0.004425	(whole milk, newspapers)	1.0	0.033914	0.420348
48	0.078579	(yogurt)	0.0	0.602200	0.361320
26	0.060758	(bottled water)	0.0	0.465628	0.279377
12	0.060399	(root vegetables)	0.0	0.462878	0.277727
22	0.052745	(shopping bags)	0.0	0.404216	0.242530

In [138…

```
uid = 1169
freq, rules, _ = mine_patterns(train_data, min_support=0.01, min_confidence=0.3, user_id=
scored = score_patterns(freq, rules=rules, data=train_data, user_id=uid, method='combinec
scored.head()
```

Out[138…

	support	itemsets	recency_score	norm_support	score
10	0.222222	(napkins)	0.327915	1.0	0.731166
37	0.111111	(cake bar, bottled beer)	1.000000	0.5	0.700000
16	0.111111	(bottled beer)	1.000000	0.5	0.700000
15	0.111111	(cake bar)	1.000000	0.5	0.700000
0	0.222222	(soda)	0.209088	1.0	0.683635

In [ ]:

```
supports  = [0.001, 0.003, 0.005, 0.01, 0.02]
confidences = [0.1, 0.3, 0.5]

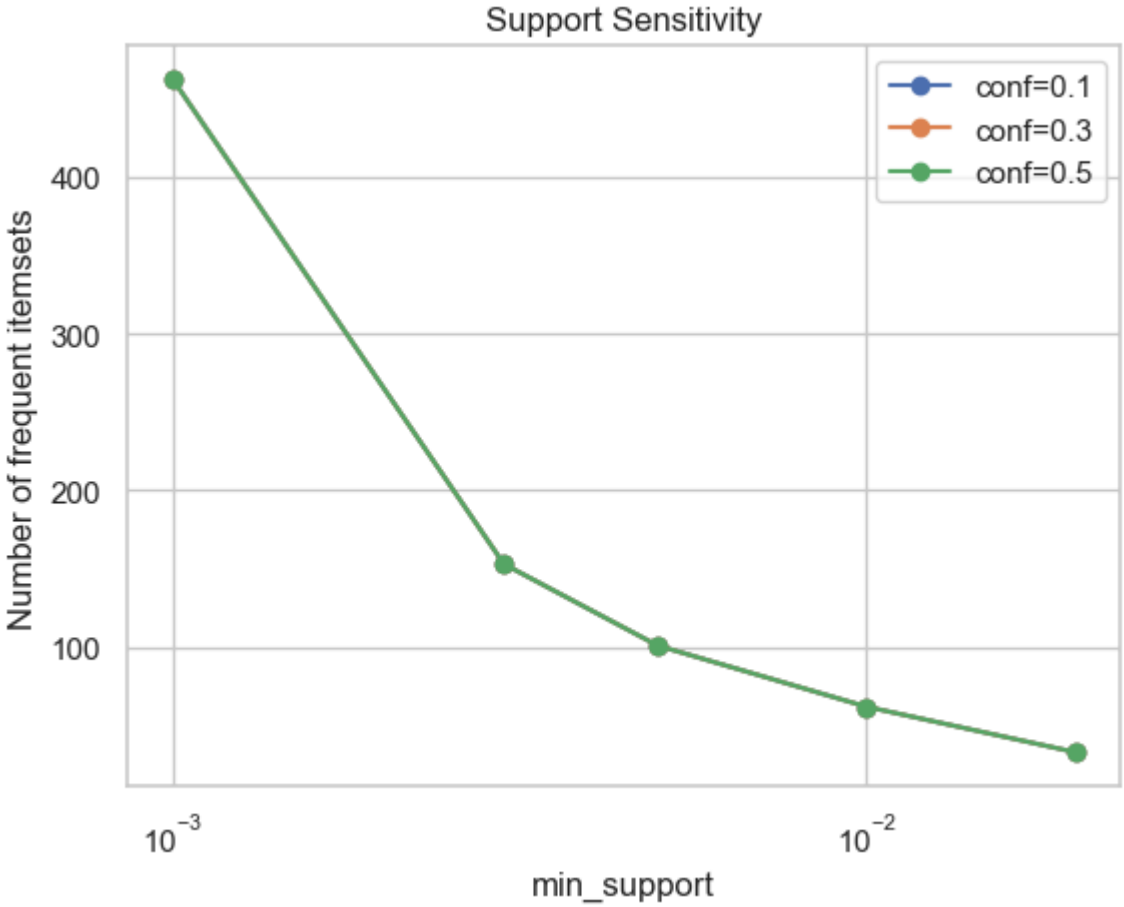
# collect stats
rows = []
for s in supports:
    for c in confidences:
        its, rules, _ = mine_patterns(train_data,
                                      min_support=s,
                                      min_confidence=c)
        rows.append({'support': s,
                     'confidence': c,
                     'n_itemsets': len(its),
                     'n_rules':   len(rules)})

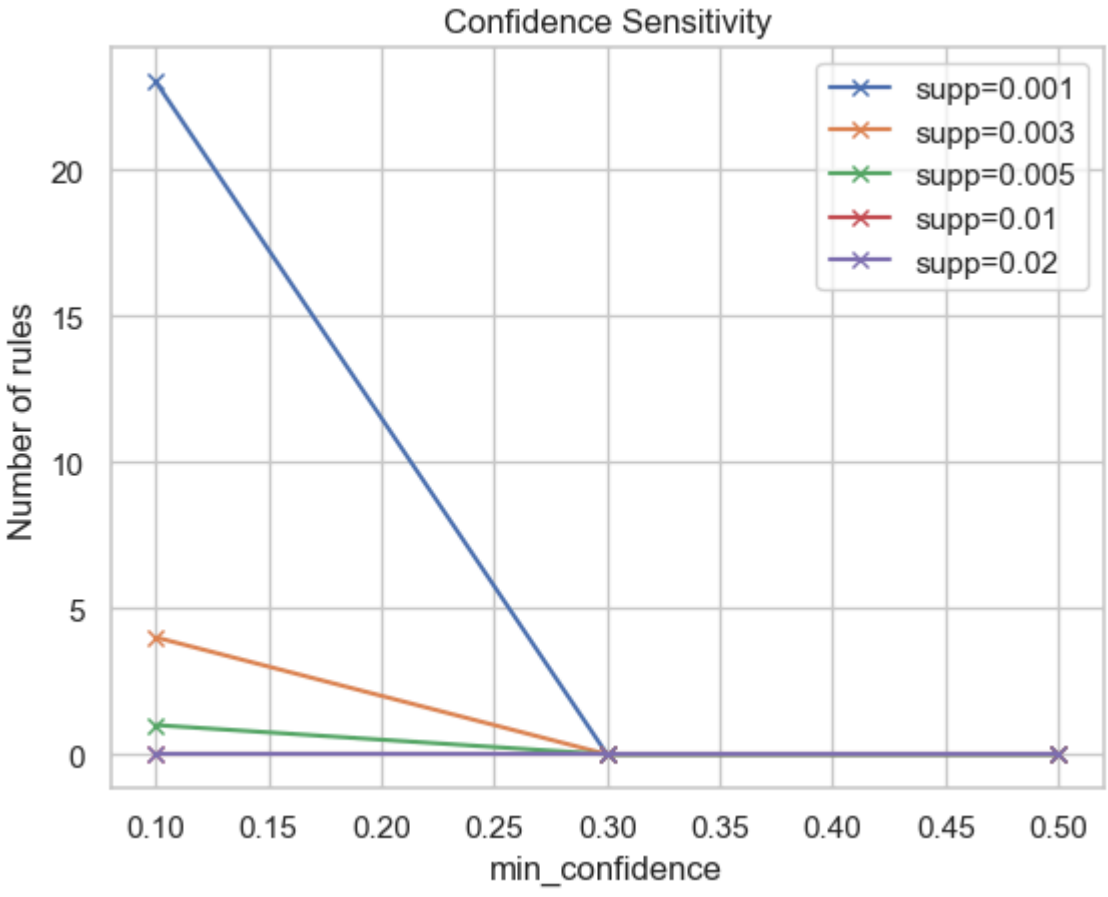
df_grid = pd.DataFrame(rows)
```

In [164…

```
# for fixed confidence- itemsets vs support
for c in confidences:
    sub = df_grid[df_grid.confidence==c]
    plt.plot(sub.support, sub.n_itemsets, marker='o', label=f'conf={c}')
plt.xscale('log')
plt.xlabel('min_support')
plt.ylabel('no. of frequent itemsets')
plt.title('support sensitivity')
plt.legend()
plt.show()
```

```
#  for a fixed support - rules vs confidence
for s in supports:
    sub = df_grid[df_grid.support==s]
    plt.plot(sub.confidence, sub.n_rules, marker='x', label=f'supp={s}')
plt.xlabel('min_confidence')
plt.ylabel('No. of rules')
plt.title('sonfidence sensitivity')
plt.legend()
plt.show()
```





In [161…

```
support = [0.1,0.01, 0.005, 0.001]
confidence = [0.07,0.1, 0.3]

results = []

for s in support:
    for c in confidence:
        freq, rules, exec_time = mine_patterns(train_data, min_support=s, min_confidence=
        print(f"Support: {s}, Confidence: {c} => Itemsets: {len(freq)}, Rules: {len(rules
        results.append({'support': s, 'confidence': c, 'itemsets': len(freq), 'rules': le
```

```
Support: 0.1, Confidence: 0.07 => Itemsets: 3, Rules: 0
Support: 0.1, Confidence: 0.1 => Itemsets: 3, Rules: 0
Support: 0.1, Confidence: 0.3 => Itemsets: 3, Rules: 0
Support: 0.01, Confidence: 0.07 => Itemsets: 62, Rules: 0
Support: 0.01, Confidence: 0.1 => Itemsets: 62, Rules: 0
Support: 0.01, Confidence: 0.3 => Itemsets: 62, Rules: 0
Support: 0.005, Confidence: 0.07 => Itemsets: 101, Rules: 9
Support: 0.005, Confidence: 0.1 => Itemsets: 101, Rules: 1
Support: 0.005, Confidence: 0.3 => Itemsets: 101, Rules: 0
Support: 0.001, Confidence: 0.07 => Itemsets: 462, Rules: 128
Support: 0.001, Confidence: 0.1 => Itemsets: 462, Rules: 23
Support: 0.001, Confidence: 0.3 => Itemsets: 462, Rules: 0
```

Observation

1. Support Sensitivity:
   - The graph represents a number of frequent itemsets by varying the support on a fixed confidence level.
   - It decreases significantly on increasing the support level.
     - Around 0.001 support, approx 400 itemsets are found.
     - Around 0.02 support, only 40(approx) itemsets are founds.

The confidence score has no significance since all three lines overlap each other. 2. Confidence Sensitivity: - The graph shows the existence of rules based on different confidence levels at fixed support values. - Most of the support values generated very few rules. - Only at support 0.001, 20+ rules are generated. - The rules drop to zero when the confidence level is above 0.3.

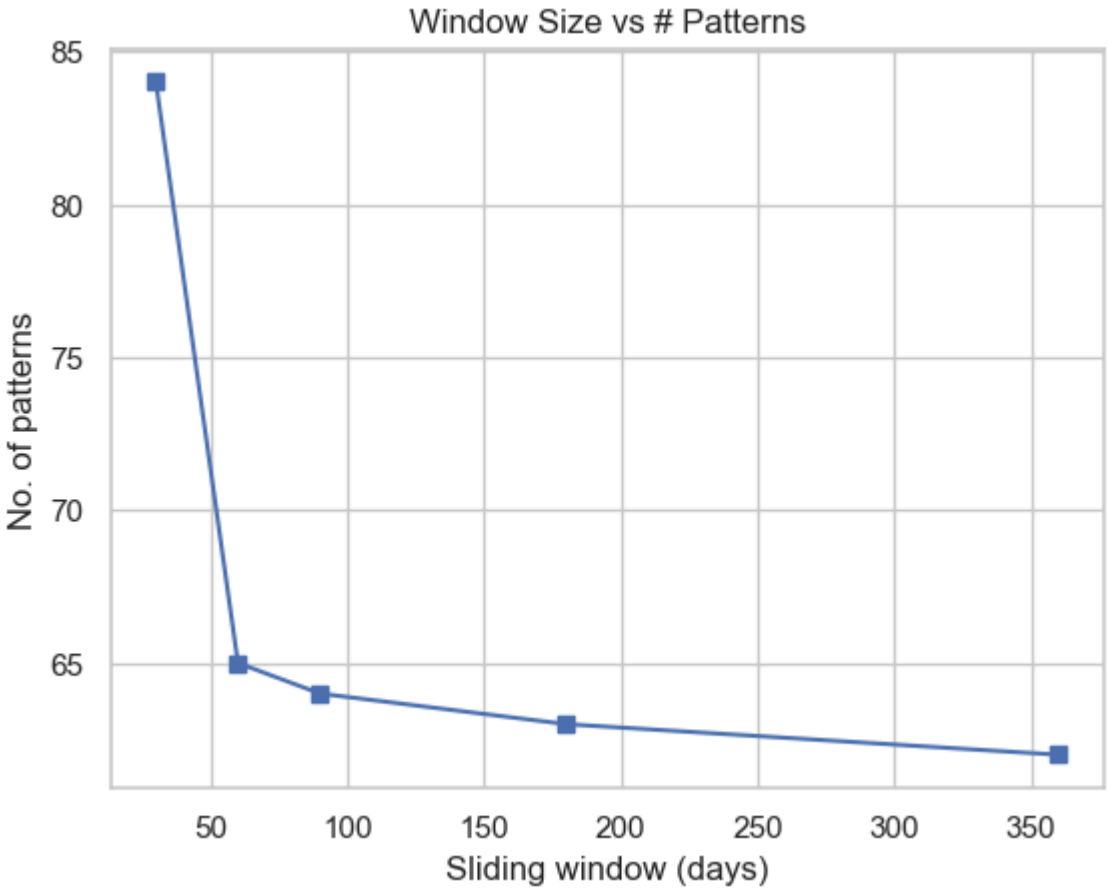
This tells us that association rule generation is very sensitive to confidence and support values and a very low support value is required to generate meaningful association.

In [170…

```
import matplotlib.pyplot as plt

windows = [30, 60, 90, 180,360]   # days
counts  = []
for W in windows:
    cutoff = train_data['Date'].max() - pd.Timedelta(days=W)
    subset = train_data[train_data['Date'] >= cutoff]
    its, rules, _ = mine_patterns(subset,
                                  min_support=0.01,
                                  min_confidence=0.3)
    counts.append(len(its))

plt.plot(windows, counts, marker='s')
plt.xlabel('Sliding window (days)')
plt.ylabel('No. of patterns')
plt.title('Window Size vs # Patterns')
plt.show()
```



Observation

This chart shows, how no. of pattern change with the size of the time period taken for the analysis at the support 0.1.

- If we take 50 days of data, it generates the maximum pattern which is up to itemsets.
- As we take older data the patterns decrease.

This analysis supports our concept of recency weighting, suggesting that recent data yields more patterns.

In [208…

```
user_id = 3681
freq, rules, _ = mine_patterns(train_data, min_support=0.005, min_confidence=0.3, user_ic

# support based
support_scored = score_patterns(freq, method='support').head(5)

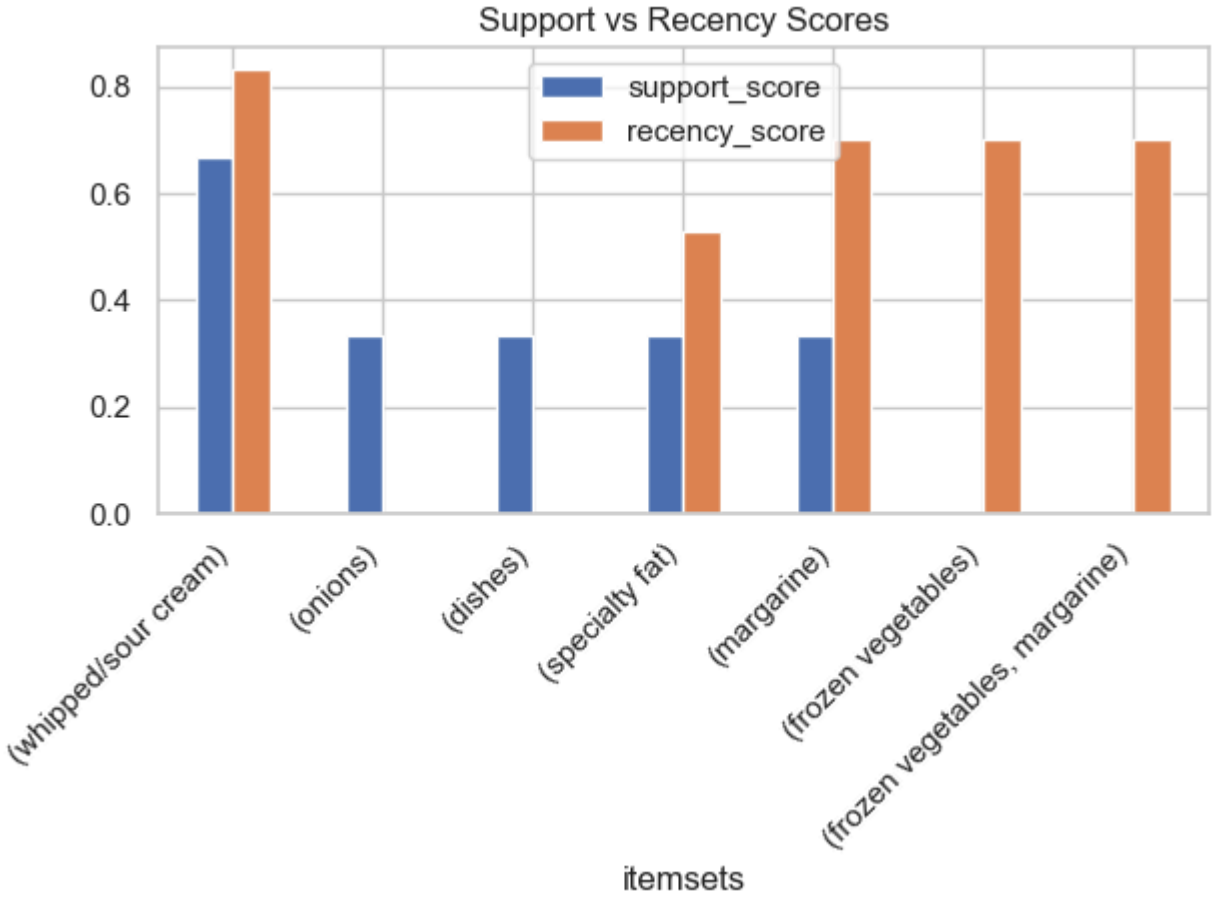
# recency-based
recency_scored = score_patterns(freq, data=train_data, user_id=user_id, method='combined'

# comparision
compare_df = support_scored[['itemsets', 'score']].rename(columns={'score':'support_score
    recency_scored[['itemsets', 'score']].rename(columns={'score':'recency_score'}),
    on='itemsets', how='outer'
).fillna(0)

print(compare_df)

# before and after plot
compare_df.plot.bar(x='itemsets', y=['support_score', 'recency_score'], title='Support vs
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

	itemsets	support_score	recency_score
0	(whipped/sour cream)	0.666667	0.83078
1	(onions)	0.333333	0.00000
2	(dishes)	0.333333	0.00000
3	(specialty fat)	0.333333	0.53078
4	(margarine)	0.333333	0.70000
5	(frozen vegetables)	0.000000	0.70000
6	(frozen vegetables, margarine)	0.000000	0.70000



Observation

- This chart compares support scoring and combined score(recency+support) on different itemsets based on the user 3681 purchasing history.
- The combined approach shows the balance between the historical frequency of items with recency.

Function for usage in Task-2

In [188…

```python
def get_pattern_recommendations(data, min_support=0.001, min_confidence=0.1,
                                mining_user_id=None, scoring_user_id=None,
                                scoring_method='combined'):
    # mine patterns globally or user specific
    frequent_itemsets, rules, _ = mine_patterns(
        data,
        min_support=min_support,
        min_confidence=min_confidence,
        user_id=mining_user_id  # May be None for global patterns
    )
    # scoring
    if not frequent_itemsets.empty:
        # use the scoring_user_id for scoring
        return score_patterns(
            frequent_itemsets,
            rules=rules,
            data=data,
```

```
            user_id=scoring_user_id,
            method=scoring_method
        )
    else:
        return pd.DataFrame()
```

- This single function can be used by the person of task 2 for mining and scoring the frequent itemsets and will help in getting personalized recommendations for the users.

In [189…
```
get_pattern_recommendations(train_data, min_support=0.001, min_confidence=0.1,mining_user
```

Out[189…

	support	itemsets	recency_score	norm_support	score
0	0.130487	(whole milk)	0.00000	1.000000	0.600000
25	0.035402	(margarine)	1.00000	0.271311	0.562786
9	0.027509	(frozen vegetables)	1.00000	0.210816	0.526489
8	0.106686	(other vegetables)	0.00000	0.817599	0.490559
3	0.101902	(rolls/buns)	0.00000	0.780935	0.468561
5	0.094606	(soda)	0.00000	0.725023	0.435014
24	0.044373	(whipped/sour cream)	0.57695	0.340055	0.434813
48	0.078579	(yogurt)	0.00000	0.602200	0.361320
26	0.060758	(bottled water)	0.00000	0.465628	0.279377
12	0.060399	(root vegetables)	0.00000	0.462878	0.277727

In [190…
```
get_pattern_recommendations(train_data, min_support=0.001, min_confidence=0.1,mining_user
```

Out[190…

	support	itemsets	recency_score	norm_support	score
12	0.250	(other vegetables)	1.0	1.0	1.0
44	0.125	(other vegetables, chocolate, waffles, rolls/b...	1.0	0.5	0.7
36	0.125	(other vegetables, rolls/buns)	1.0	0.5	0.7
37	0.125	(other vegetables, chocolate, rolls/buns)	1.0	0.5	0.7
38	0.125	(waffles, rolls/buns)	1.0	0.5	0.7
39	0.125	(chocolate, waffles)	1.0	0.5	0.7
40	0.125	(other vegetables, waffles)	1.0	0.5	0.7
18	0.125	(waffles)	1.0	0.5	0.7
35	0.125	(chocolate, rolls/buns)	1.0	0.5	0.7
16	0.125	(chocolate)	1.0	0.5	0.7

Significance of Analysis

- Provide a general overview of data.
- Explain various thresholds for mining meaningful frequent sets.
- Explains time factor in data mining.
- Parameter Optimization: Provide insights on tuning the values of pattern miner for the recommender.

Resource	In-text citation	Reference
Recency-Weighted Scoring	(Customers.ai, 2025)	Customers.ai, 2025. Recency-Weighted Scoring. https://customers.ai/recer
Implement FP Growth Algorithm in Python	(Aditya, 2023)	Aditya, 2023. Implement FP Growth Algorithm in Python. https://codinginf algorithm-in-python/
Cracking the metrics for pattern mining	(Sharma, 2024)	Sharma, R., 2024. Cracking the metrics for pattern mining using Associatio https://medium.com/@rakeshsharma.pr/cracking-the-metrics-for-pattern- 51acab7a0126
mlxtend documentation	(mlxtend, 2025)	mlxtend, 2025. Welcome to mlxtend's documentation! https://rasbt.github
Dickey-Fuller test for Time Series Stationarity	(Bhatt, 2025)	Bhatt, B., 2025. Dickey-Fuller test for Time Series Stationarity using Python v=warCSvy1DMk
Association Rules: Apriori and FP-Growth	(HackerTraining, 2025)	HackerTraining, 2025. Association Rules: Apriori and FP-Growth. https://hackertraining.org/ai_research/ML_Fundamentals/ai_generated/da Growth%29/

Grocery Recommendation System - Collaborative Filtering

The Collaborative Filtering task is connected to both Task 1 and the raw input from the dataset. In the first half of the notebook, we will try to cover the application of collaborative filtering to the raw grocery dataset. In the second half, we will cover the integration of Task 1 with CF and the evaluation metric used.

Initial Steps

In [4]:
```
# Importing all relevant packages here
import pandas as pd
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity
import matplotlib.pyplot as plt

import warnings
warnings.filterwarnings("ignore")
```

In [5]:
```
# Reading the CSV Training dataset
# We also create a new column called date for recency weighted recommendation

dftr = pd.read_csv('Groceries data train.csv')
dftr['date'] = pd.to_datetime(dftr['Date'], dayfirst=True)
dfte = pd.read_csv('Groceries data test.csv')
dfte['date'] = pd.to_datetime(dfte['Date'], dayfirst=True)
```

In [6]:
```
dftr.head()
```

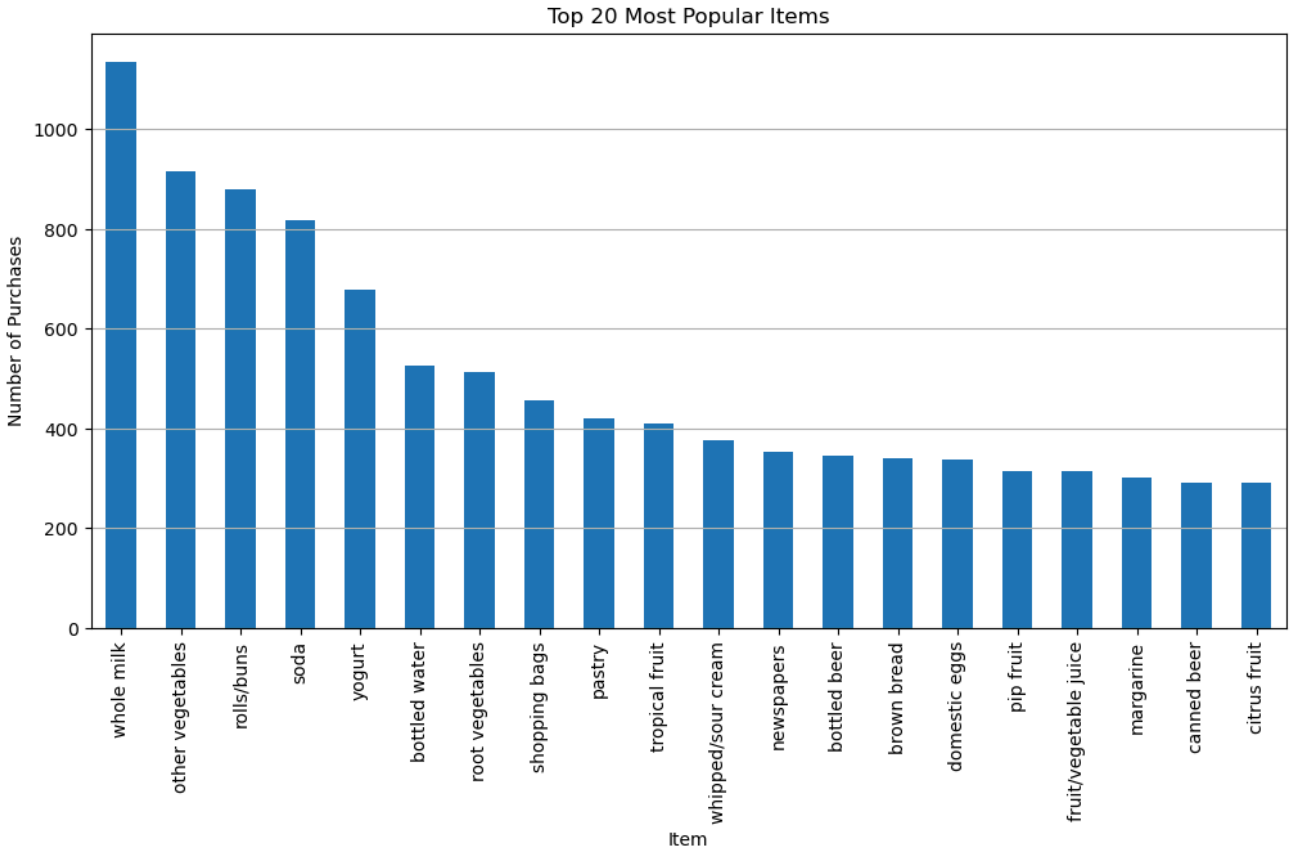
Out[6]:

	User_id	Date	itemDescription	year	month	day	day_of_week	date
0	2351.0	1/01/2014	cleaner	2014.0	1.0	1.0	2.0	2014-01-01
1	2226.0	1/01/2014	sausage	2014.0	1.0	1.0	2.0	2014-01-01
2	1922.0	1/01/2014	tropical fruit	2014.0	1.0	1.0	2.0	2014-01-01
3	2943.0	1/01/2014	whole milk	2014.0	1.0	1.0	2.0	2014-01-01
4	1249.0	1/01/2014	citrus fruit	2014.0	1.0	1.0	2.0	2014-01-01

Exploratory Data Analysis

In [11]:
```
# The bar plot shows the top most bought items in the dataset. This is relevant because t
# top 5 recommendations for users not found in the dataset.

top = dftr['itemDescription'].value_counts().head(20)
plt.figure(figsize=(12, 6))
top.plot(kind='bar')
plt.title('Top 20 Most Popular Items')
plt.xlabel('Item')
plt.ylabel('Number of Purchases')
plt.xticks(rotation=90)
plt.grid(axis='y')
plt.show()
```

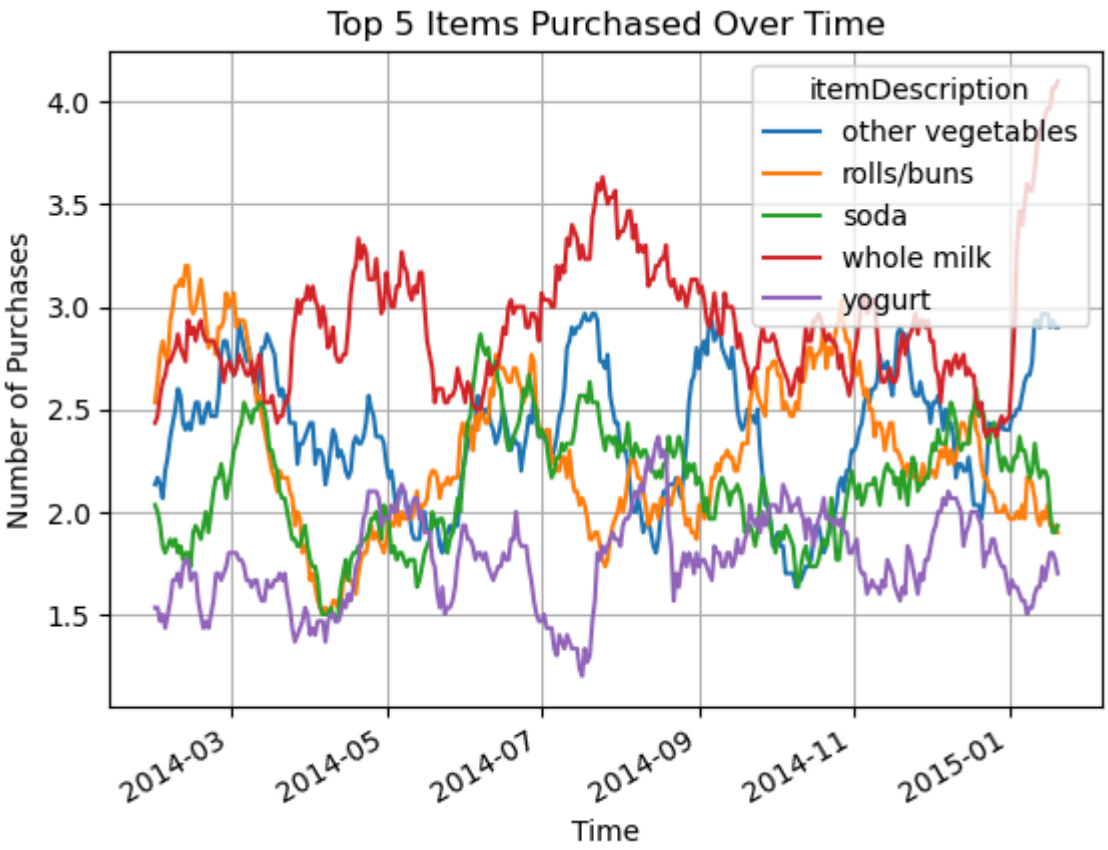


In [21]:
```
#  We can see from the graph that the items are bought very heavily at certain time perio
#  of Recency weighting to out CF method for providing recommendations.

tt = dftr['itemDescription'].value_counts().head(5).index
ttd = dftr[dftr['itemDescription'].isin(tt)]
tttr = ttd.groupby(['date', 'itemDescription']).size().unstack(fill_value=0).rolling(winc

plt.figure(figsize=(14, 7))
tttr.plot()
plt.title('Top 5 Items Purchased Over Time')
plt.xlabel('Time')
plt.ylabel('Number of Purchases')
plt.grid(True)
plt.show()
```

```
<Figure size 1400x700 with 0 Axes>
```



Implementation

Creating a Utility Matrix of Users and Items

In [12]:
```
# The first step in the CF process is to create the user item matrix or utility matrix cc
# user to the item

def create_useritem_matrix(decay=0.05):
    max_date = dftr['date'].max()
    dftr['days_diff'] = (max_date - dftr['date']).dt.days
    dftr['recency_weight'] = np.exp(-decay * dftr['days_diff'])
    utility_df = dftr.groupby(["User_id","itemDescription"]).size().unstack(fill_value=0)
    utility_binary = (utility_df > 0).astype(int)
    weighted_mat = dftr.groupby(['User_id', 'itemDescription'])['recency_weight'].sum().u
    return utility_binary, weighted_mat
```

Compute Similarity Matrix

The following three code blocks calculates the cosine similarity using the sklearn library for user-user item-item and an adjusted cosine similarity function which tries to implement the formula provided in the research paper.

In [15]:
```
def compute_cosine_similarity_items(utilitymatrix):
    itemsim = cosine_similarity(utilitymatrix.T)
    df_item = pd.DataFrame(itemsim, index=utilitymatrix.columns, columns=utilitymatrix.cc

    return df_item
```

In [17]:
```
def compute_cosine_similarity_users(utilitymatrix):
    usersim = cosine_similarity(utilitymatrix)
    df_user = pd.DataFrame(usersim, index=utilitymatrix.index, columns=utilitymatrix.inde
    return df_user
```

In [19]:
```
def compute_adjusted_cosine_similarity_items(utilitymatrix):
    usermean = utilitymatrix.mean(axis=1)
    utilitymatrixtemp = utilitymatrix.sub(usermean, axis=0)
    itemsim = cosine_similarity(utilitymatrixtemp.T)
    df_item_adj = pd.DataFrame(itemsim, index=utilitymatrix.columns, columns=utilitymatri
    return df_item_adj
```

Generate Top 5 recommendations

In [22]:
```
# The function below is used to create all the necessary items to use for recommendation.
# we are separating it into this function
def initilaise_recommendation():
    useritem,weightedmat = create_useritem_matrix()
    df_item = compute_cosine_similarity_items(useritem)
    df_item_adj = compute_adjusted_cosine_similarity_items(useritem)
    df_user = compute_cosine_similarity_users(useritem)
    return useritem, weightedmat, df_item, df_item_adj, df_user

useritem, weightedmat, df_item, df_item_adj, df_user = initilaise_recommendation()
```

In [24]:
```
# This function is where the all of the CF algorithm integrates. The cold start for a use
# Otherwise, the top scores from user-user item-item and item adjusted results are return
# The scoring for item-item matrix works as a dot product of the row of recency weighted
# Now before we return the top scored items, we will filter already purchased items by th

def recommendation_top5(user_id):
    if user_id not in useritem.index:
        try:
            user_id = int(user_id)
        except ValueError:
            return useritem.sum().nlargest(5).index.tolist()
    if user_id not in useritem.index:
        return useritem.sum().nlargest(5).index.tolist()

    sims = df_user.loc[user_id].copy()
    sims[user_id] = 0.0
    scores_user = weightedmat.T.dot(sims)
    already = useritem.loc[user_id]
    scores_user.drop(already[already > 0].index, inplace=True)

    u_vec = weightedmat.loc[user_id]
    scores = u_vec.dot(df_item)
    purchased = u_vec[u_vec > 0].index
    scores = scores.drop(purchased, errors='ignore')

    u_vec2 = weightedmat.loc[user_id]
    scores_itemadj = u_vec2.dot(df_item_adj)
    purchased2 = u_vec2[u_vec2 > 0].index
    scores_itemadj = scores_itemadj.drop(purchased2, errors='ignore')


    top5item  = scores.nlargest(5).index.tolist()
    top5itemadj  = scores_itemadj.nlargest(5).index.tolist()
    top5user  = scores_user.nlargest(5).index.tolist()
    return top5item, top5user, top5itemadj
```

In [26]:
```
print("Recommendations for User 2351:", recommendation_top5(2351.0))
```

```
Recommendations for User 2351: (['soda', 'whole milk', 'other vegetables', 'yogurt', 'roll
s/buns'], ['whole milk', 'other vegetables', 'soda', 'rolls/buns', 'root vegetables'], ['p
rocessed cheese', 'mayonnaise', 'cling film/bags', 'butter milk', 'mustard'])
```

Testing and Evaluation

The metrics of Precision, Recall, F1 and MAP score was selected. In this section of the code, you can see additional scores like RMSE or MAE which indicates from the final graph that the User based recommendation performed well but is not stable, with the raw scoring.

In [29]:
```
def evaluate_cf():
    actual = dfte.groupby('user_id')['itemDescription'].apply(set).to_dict()
    records = []
    for uid, true_items in actual.items():
        if uid not in useritem.index:
            continue
        rec_item, rec_user, rec_item_adj = recommendation_top5(uid)

        def rank_metrics(recs, true_set):
            hits = 0
            for item in recs:
                if item in true_set:
                    hits += 1
            precision = hits / 5
            if len(true_set) > 0:
                recall = hits / len(true_set)
            else:
                recall = np.nan
            if (precision + recall) > 0:
                f1 = 2 * precision * recall / (precision + recall)
            else:
                f1 = 0.0
            ap = 0.0
            hit_count = 0
            for k in range(1, 6):
                candidate = recs[k-1]
                if candidate in true_set:
                    hit_count += 1
                    ap += hit_count / k

            if len(true_set) > 0:
                denom = min(len(true_set), 5)
                ap = ap / denom
            else:
                ap = np.nan
            return precision, recall, f1, ap

        p_i, r_i, f_i, m_i = rank_metrics(rec_item, true_items)
        p_u, r_u, f_u, m_u = rank_metrics(rec_user, true_items)

        # rating metrics on positives
        ps_i = weightedmat.loc[uid].dot(df_item)
        errs_i = [abs(ps_i[i] - 1) for i in true_items]
        mae_i = np.mean(errs_i) if errs_i else np.nan
        rmse_i = np.sqrt(np.mean([(ps_i[i] - 1)**2 for i in true_items])) if errs_i else

        sims = df_user.loc[uid].copy(); sims[uid] = 0
        ps_u = weightedmat.T.dot(sims)
        errs_u = [abs(ps_u[i] - 1) for i in true_items]
        mae_u = np.mean(errs_u) if errs_u else np.nan
        rmse_u = np.sqrt(np.mean([(ps_u[i] - 1)**2 for i in true_items])) if errs_u else

        records.append({'user_id': uid,'precision_item': p_i, 'recall_item': r_i, 'f1_ite
            'precision_user': p_u, 'recall_user': r_u, 'f1_user': f_u, 'map_user': m_u, '

    df_res = pd.DataFrame(records)
    summary = df_res.mean(numeric_only=True)
```

```
    print("=== Evaluation Summary ===")
    print(f"Item-based  — Prec@5:{summary['precision_item']:.4f}, Rec@5:{summary['recall_
    print(f"User-based  — Prec@5:{summary['precision_user']:.4f}, Rec@5:{summary['recall_
    return df_res

results = evaluate_cf()
```

```
=== Evaluation Summary ===
Item-based  — Prec@5:0.2067, Rec@5:0.1944, F1:0.1836, MAP:0.1639, MAE:0.9683, RMSE:0.9699
User-based  — Prec@5:0.2214, Rec@5:0.2072, F1:0.1966, MAP:0.1722, MAE:2.0547, RMSE:2.8489
```

In [30]:
```
results.head()
```

Out[30]:

	user_id	precision_item	recall_item	f1_item	map_item	mae_item	rmse_item	precision_user
0	1000	0.4	0.222222	0.285714	0.166667	0.999988	0.999988	0.2
1	1001	0.0	0.000000	0.000000	0.000000	0.644973	0.692928	0.0
2	1002	0.0	0.000000	0.000000	0.000000	0.999828	0.999828	0.2
3	1003	0.0	0.000000	0.000000	0.000000	0.998411	0.998412	0.0
4	1004	0.0	0.000000	0.000000	0.000000	0.990822	0.990833	0.0

In [37]:
```
# Plotting a bar graph of the performance metrics

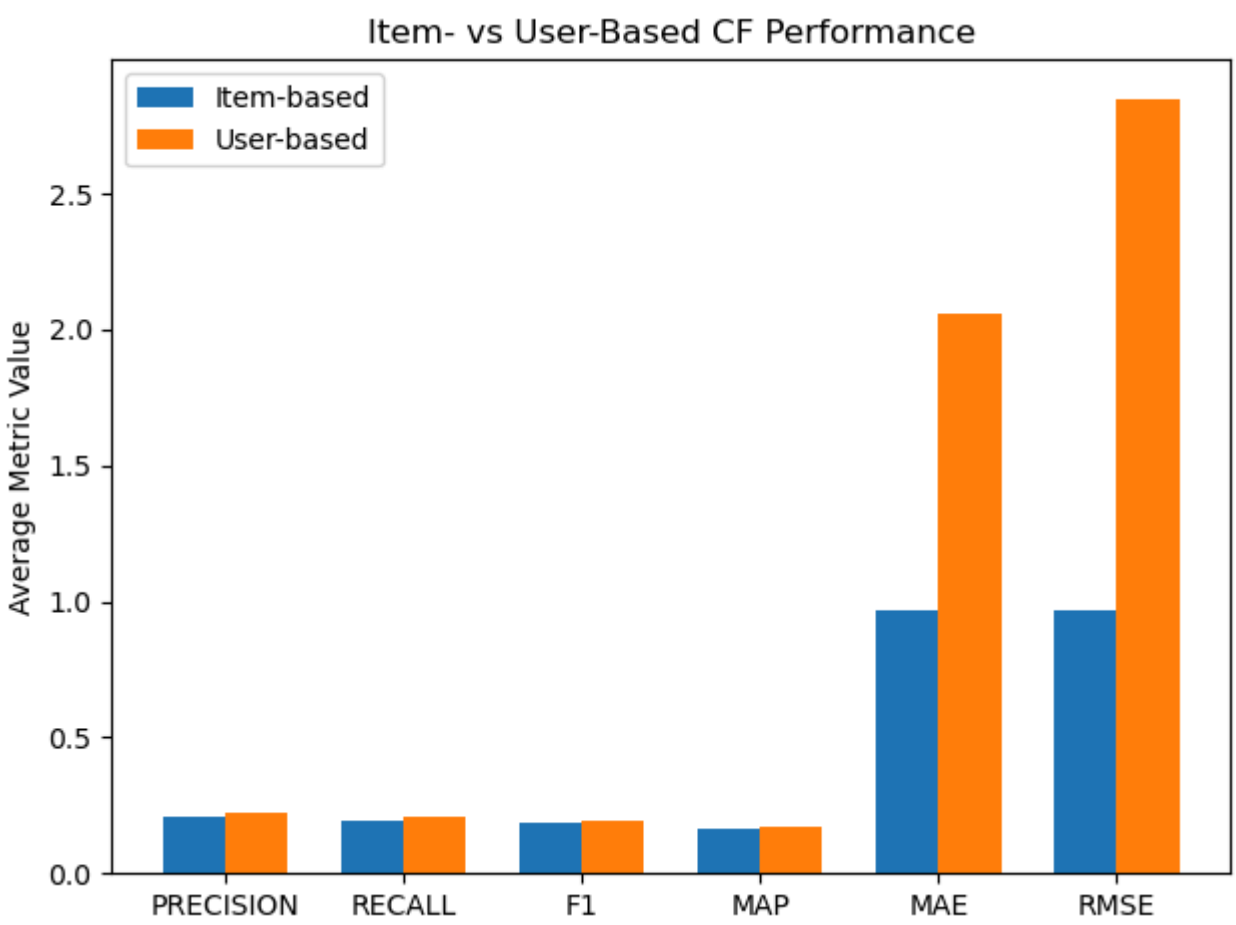
metrics = ['precision', 'recall', 'f1', 'map', 'mae', 'rmse']

item_means = [results[f'{m}_item'].mean() for m in metrics]
user_means = [results[f'{m}_user'].mean() for m in metrics]

x = np.arange(len(metrics))
width = 0.35
fig, ax = plt.subplots()
ax.bar(x - width/2, item_means, width, label='Item-based')
ax.bar(x + width/2, user_means, width, label='User-based')

ax.set_xticks(x)
ax.set_xticklabels([m.upper() for m in metrics])
ax.set_ylabel('Average Metric Value')
ax.set_title('Item- vs User-Based CF Performance')
ax.legend()

plt.tight_layout()
plt.show()
```



Results for Adjusted Cosine Similarity

This section of the code is to evaluate the adjusted cosine similarity metric and its performance of this dataset. The output indicates that the Adjusted Cosine Similarity metric did not fair well.

In [40]:
```
def recommendation_top5_adj(user_id):
    if user_id not in useritem.index:
        try:
            user_id = int(user_id)
        except ValueError:
            return useritem.sum().nlargest(5).index.tolist()
    if user_id not in useritem.index:
        return useritem.sum().nlargest(5).index.tolist()

    sims = df_user.loc[user_id].copy()
    sims[user_id] = 0.0
    scores_user = weightedmat.T.dot(sims)
    already = useritem.loc[user_id]
    scores_user.drop(already[already > 0].index, inplace=True)

    u_vec = weightedmat.loc[user_id]
    scores = u_vec.dot(df_item)
    purchased = u_vec[u_vec > 0].index
    scores = scores.drop(purchased, errors='ignore')

    u_vec2 = weightedmat.loc[user_id]
    scores_itemadj = u_vec2.dot(df_item_adj)
    purchased2 = u_vec2[u_vec2 > 0].index
    scores_itemadj = scores_itemadj.drop(purchased2, errors='ignore')


    top5item  = scores.nlargest(5).index.tolist()
    top5itemadj  = scores_itemadj.nlargest(5).index.tolist()
```

```
    top5user  = scores_user.nlargest(5).index.tolist()
    return top5itemadj, top5user, top5item
```

In [44]:

```
def evaluate_cf_adj():
    actual = dfte.groupby('user_id')['itemDescription'].apply(set).to_dict()
    records = []
    for uid, true_items in actual.items():
        if uid not in useritem.index:
            continue
        rec_item, rec_user, rec_item_adj = recommendation_top5_adj(uid)

        def rank_metrics(recs, true_set):
            hits = 0
            for item in recs:
                if item in true_set:
                    hits += 1
            precision = hits / 5
            if len(true_set) > 0:
                recall = hits / len(true_set)
            else:
                recall = np.nan
            if (precision + recall) > 0:
                f1 = 2 * precision * recall / (precision + recall)
            else:
                f1 = 0.0
            ap = 0.0
            hit_count = 0
            for k in range(1, 6):
                candidate = recs[k-1]
                if candidate in true_set:
                    hit_count += 1
                    ap += hit_count / k

            if len(true_set) > 0:
                denom = min(len(true_set), 5)
                ap = ap / denom
            else:
                ap = np.nan
            return precision, recall, f1, ap

        p_i, r_i, f_i, m_i = rank_metrics(rec_item, true_items)
        p_u, r_u, f_u, m_u = rank_metrics(rec_user, true_items)

        # rating metrics on positives
        ps_i = weightedmat.loc[uid].dot(df_item)
        errs_i = [abs(ps_i[i] - 1) for i in true_items]
        mae_i = np.mean(errs_i) if errs_i else np.nan
        rmse_i = np.sqrt(np.mean([(ps_i[i] - 1)**2 for i in true_items])) if errs_i else

        sims = df_user.loc[uid].copy(); sims[uid] = 0
        ps_u = weightedmat.T.dot(sims)
        errs_u = [abs(ps_u[i] - 1) for i in true_items]
        mae_u = np.mean(errs_u) if errs_u else np.nan
        rmse_u = np.sqrt(np.mean([(ps_u[i] - 1)**2 for i in true_items])) if errs_u else

        records.append({'user_id': uid,'precision_item': p_i, 'recall_item': r_i, 'f1_ite
            'precision_user': p_u, 'recall_user': r_u, 'f1_user': f_u, 'map_user': m_u, '

    df_res = pd.DataFrame(records)
    summary = df_res.mean(numeric_only=True)
    print("=== Evaluation Summary ===")
    print(f"Item-based  — Prec@5:{summary['precision_item']:.4f}, Rec@5:{summary['recall_
    print(f"User-based  — Prec@5:{summary['precision_user']:.4f}, Rec@5:{summary['recall_
    return df_res
```

```
results2 = evaluate_cf_adj()
```

```
=== Evaluation Summary ===
Item-based  — Prec@5:0.1642, Rec@5:0.1528, F1:0.1450, MAP:0.1282, MAE:0.9683, RMSE:0.9699
User-based  — Prec@5:0.2214, Rec@5:0.2072, F1:0.1966, MAP:0.1722, MAE:2.0547, RMSE:2.8489
```

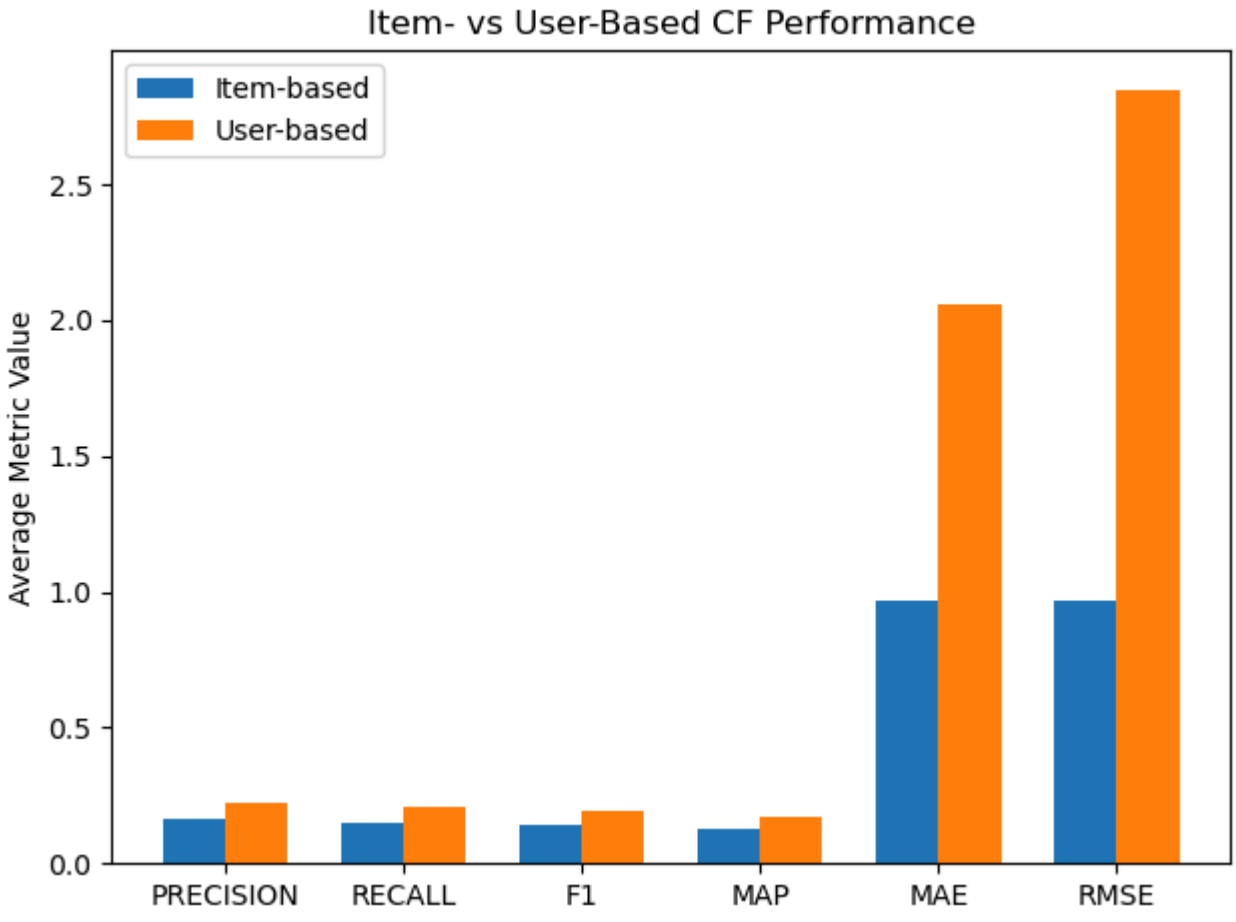
In [46]:
```
metrics = ['precision', 'recall', 'f1', 'map', 'mae', 'rmse']

item_means = [results2[f'{m}_item'].mean() for m in metrics]
user_means = [results2[f'{m}_user'].mean() for m in metrics]

x = np.arange(len(metrics))
width = 0.35
fig, ax = plt.subplots()
ax.bar(x - width/2, item_means, width, label='Item-based')
ax.bar(x + width/2, user_means, width, label='User-based')

ax.set_xticks(x)
ax.set_xticklabels([m.upper() for m in metrics])
ax.set_ylabel('Average Metric Value')
ax.set_title('Item- vs User-Based CF Performance')
ax.legend()

plt.tight_layout()
plt.show()
```



Integrated Code

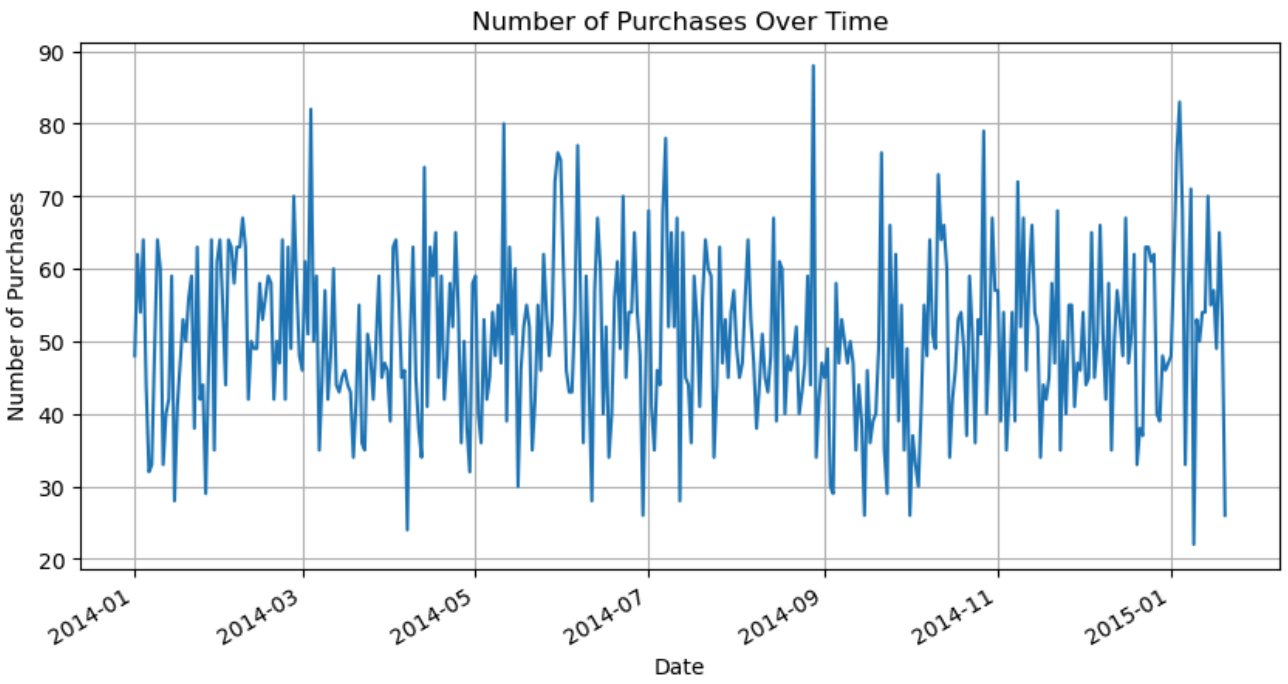
Exploratory Data Analysis

In [3]:
```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import warnings
warnings.filterwarnings("ignore")
df = pd.read_csv('Groceries data train.csv')
df['Date'] = pd.to_datetime(df['Date'], dayfirst=True)
df.head()
```

Out[3]:

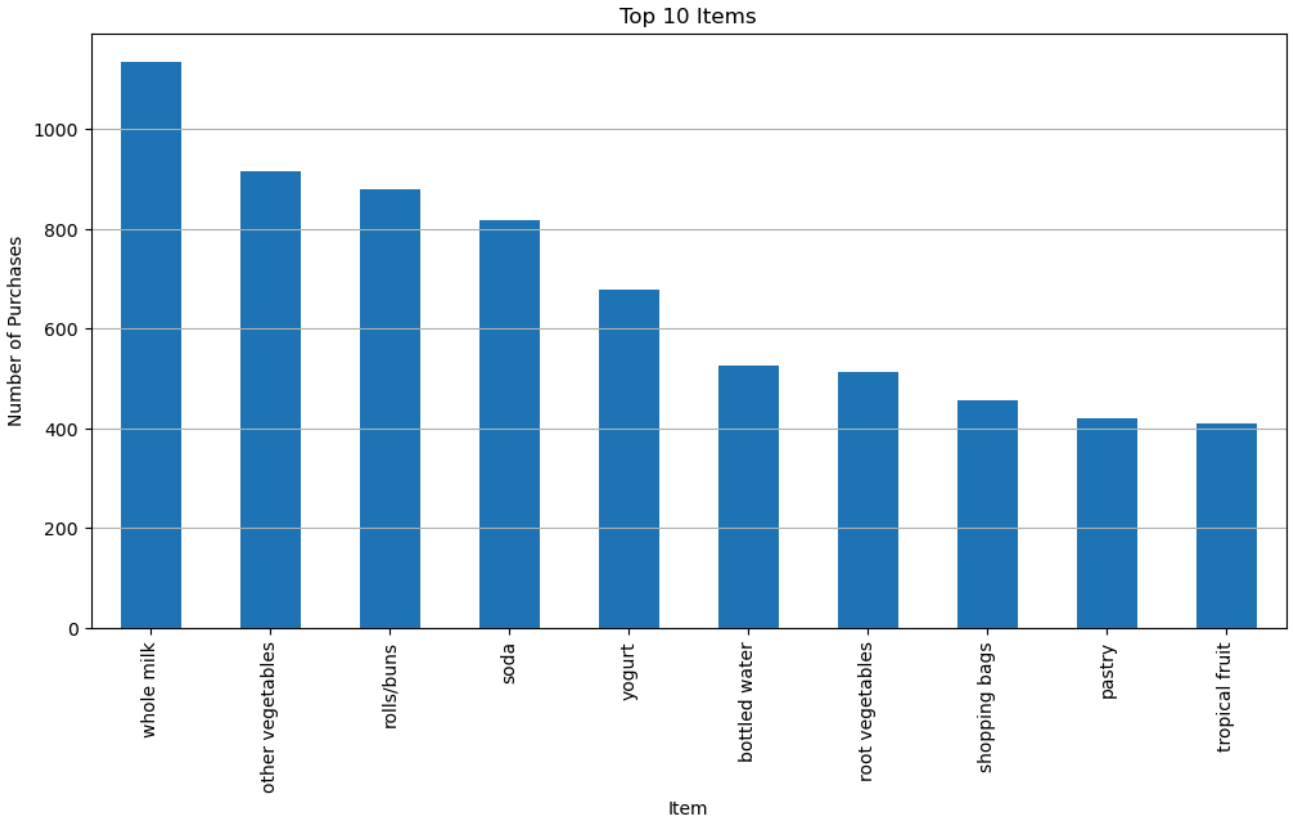
	User_id	Date	itemDescription	year	month	day	day_of_week
0	2351.0	2014-01-01	cleaner	2014.0	1.0	1.0	2.0
1	2226.0	2014-01-01	sausage	2014.0	1.0	1.0	2.0
2	1922.0	2014-01-01	tropical fruit	2014.0	1.0	1.0	2.0
3	2943.0	2014-01-01	whole milk	2014.0	1.0	1.0	2.0
4	1249.0	2014-01-01	citrus fruit	2014.0	1.0	1.0	2.0

In [4]:
```
purchase = df.groupby('Date').size()
plt.figure(figsize=(10, 5))
purchase.plot()
plt.title('Number of Purchases Over Time')
plt.xlabel('Date')
plt.ylabel('Number of Purchases')
plt.grid(True)
plt.show()
```



In [7]:
```
tt = df['itemDescription'].value_counts().head(10)
plt.figure(figsize=(12, 6))
tt.plot(kind='bar')
plt.title('Top 10 Items')
plt.xlabel('Item')
```

```
plt.ylabel('Number of Purchases')
plt.xticks(rotation=90)
plt.grid(axis='y')
plt.show()
```



Combined Code

Task 1 Associative Pattern Mining

In [11]:
```
from mlxtend.frequent_patterns import fpgrowth, association_rules
from datetime import datetime
import time
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
import numpy as np

train_data = pd.read_csv('Groceries data train.csv')
train_data['Date'] = pd.to_datetime(train_data['Date'], format='mixed')
```

In [12]:
```
def prepare_transactions(data, user_id=None):
    if user_id is not None:
        data = data[data['User_id'] == user_id]
        if data.empty:
            return None
    transactions = data.groupby(['User_id', 'Date'])['itemDescription'].apply(list).reset
    item_lists = transactions['itemDescription'].tolist()
    te = TransactionEncoder()
    te_ary = te.fit(item_lists).transform(item_lists)
    encoded_df = pd.DataFrame(te_ary, columns=te.columns_)
    encoded_df.insert(0, 'Date', transactions['Date'])
    encoded_df.insert(0, 'User_id', transactions['User_id'])
    encoded_df.set_index(['User_id', 'Date'], inplace=True)
    return encoded_df
```

In [13]:
```
# Function to mine patterns using FP-Growth
def mine_patterns(data, min_support=0.01, max_length=5, min_confidence=0.3, user_id=None)
    start_time = time.time()
```

```
    # Prepare transactions
    transactions = prepare_transactions(data, user_id)
    if transactions is None or transactions.empty:
        print("No transactions found.")
        return pd.DataFrame(), pd.DataFrame(), time.time() - start_time
    # Mine frequent itemsets using FP-Growth
    try:
        # Convert to boolean
        items_df = transactions.astype(bool)
        # Apply FP-Growth algorithm
        frequent_itemsets = fpgrowth(
            items_df,
            min_support=min_support,
            max_len=max_length,
            use_colnames=True
        )
        # Generate association rules
        if not frequent_itemsets.empty:
            rules = association_rules(
                frequent_itemsets,
                metric='confidence',
                min_threshold=min_confidence
            )
        else:
            rules = pd.DataFrame()
    except Exception as e:
        print(f"Error mining patterns: {e}")
        frequent_itemsets = pd.DataFrame()
        rules = pd.DataFrame()

    execution_time = time.time() - start_time
    return frequent_itemsets, rules, execution_time

# frequent_itemsets, rules, _ = mine_patterns(train_data, min_support=0.001, min_confiden
```

In [17]:
```
def calculate_recency_weights(user_data):
    user_data = user_data.copy()
    user_data['DateConverted'] = pd.to_datetime(user_data['Date'])
    # most recent purchase
    max_date = user_data['DateConverted'].max()
    recency_weights = {}
    for item in user_data['itemDescription'].unique():
        # dates when this item was purchased
        item_dates = user_data[user_data['itemDescription'] == item]['DateConverted']
        if not item_dates.empty:
            #  most recent purchase date for this item
            most_recent = item_dates.max()
            # calc. days since most recent purchase
            days_since = (max_date - most_recent).days
            # recency weight with decay rate of 0.05
            recency_weights[item] = np.exp(-0.005 * days_since)
            # Decay rate of 0.05
    return recency_weights
```

In [19]:
```
# function to calculate pattern recency
def calculate_pattern_recency(itemset, recency_weights):
    weights = [recency_weights.get(item, 0) for item in itemset]
    if not weights:
        return 0
    return sum(weights) / len(weights)
```

In [21]:
```
def score_patterns(frequent_itemsets, rules=None, data=None, user_id=None, method='combir
    if frequent_itemsets.empty:
        return pd.DataFrame()
    scored_patterns = frequent_itemsets.copy()
```

```
    # scoring based on support
    if method == 'support':
        scored_patterns['score'] = scored_patterns['support']

    # if rules and want confidence based scoring
    elif method == 'confidence' and rules is not None and not rules.empty:
        itemset_scores = {}
        for _, rule in rules.iterrows():
            itemset = frozenset(rule['antecedents'].union(rule['consequents']))
            score = rule['confidence']
            if itemset in itemset_scores:
                itemset_scores[itemset] = max(itemset_scores[itemset], score)
            else:
                itemset_scores[itemset] = score
        scored_patterns['score'] = scored_patterns['itemsets'].apply(lambda x: itemset_sc
    # recency based scoring (transaction data and user_id)
    elif method == 'recency' and data is not None and user_id is not None:
        user_data = data[data['User_id'] == user_id]
        if user_data.empty: # support scoring if no user data
            scored_patterns['score'] = scored_patterns['support']
        else:
            recency_weights = calculate_recency_weights(user_data)
            scored_patterns['score'] = scored_patterns['itemsets'].apply(lambda itemset:
    #  using both support and recency
    elif method == 'combined' and data is not None and user_id is not None:
        user_data = data[data['User_id'] == user_id]
        if user_data.empty:
            scored_patterns['score'] = scored_patterns['support']
        else:
            recency_weights = calculate_recency_weights(user_data)
            scored_patterns['recency_score'] = scored_patterns['itemsets'].apply(lambda i
            max_support = scored_patterns['support'].max()
            if max_support > 0:
                scored_patterns['norm_support'] = scored_patterns['support'] / max_suppor
            else:
                scored_patterns['norm_support'] = scored_patterns['support']
            scored_patterns['score'] = 0.6 * scored_patterns['norm_support'] + 0.4 * scor
    #default to support
    else:
        scored_patterns['score'] = scored_patterns['support']

    return scored_patterns.sort_values('score', ascending=False)

# frequent_itemsets, rules, _ = mine_patterns(train_data, min_support=0.001, min_confiden
# scored_patterns = score_patterns(frequent_itemsets, rules=rules, data=train_data, user_
# scored_patterns.head(10)
```

Task 2 Collaborative Filtering

In [24]:
```
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.preprocessing import normalize

dftr = pd.read_csv('Groceries data train.csv')
dftr['date'] = pd.to_datetime(dftr['Date'], dayfirst=True)
```

In [26]:
```
def create_useritem_matrix(decay=0.05):
    max_date = dftr['date'].max()
    dftr['days_diff'] = (max_date - dftr['date']).dt.days
    dftr['recency_weight'] = np.exp(-decay * dftr['days_diff'])
    utility_df = dftr.groupby(["User_id","itemDescription"]).size().unstack(fill_value=0)
    utility_binary = (utility_df > 0).astype(int)
    weighted_mat = dftr.groupby(['User_id', 'itemDescription'])['recency_weight'].sum().u
    return utility_binary, weighted_mat
```

In [28]:
```
def compute_cosine_similarity_items(utilitymatrix):
    itemsim = cosine_similarity(utilitymatrix.T)
    df_item = pd.DataFrame(itemsim, index=utilitymatrix.columns, columns=utilitymatrix.cc

    return df_item

def compute_cosine_similarity_users(utilitymatrix):
    usersim = cosine_similarity(utilitymatrix)
    df_user = pd.DataFrame(usersim, index=utilitymatrix.index, columns=utilitymatrix.inde
    return df_user

def compute_adjusted_cosine_similarity_items(utilitymatrix):
    usermean = utilitymatrix.mean(axis=1)
    utilitymatrixtemp = utilitymatrix.sub(usermean, axis=0)
    itemsim = cosine_similarity(utilitymatrixtemp.T)
    df_item_adj = pd.DataFrame(itemsim, index=utilitymatrix.columns, columns=utilitymatri
    return df_item_adj
```

In [30]:
```
def initilaise_recommendation():
    useritem,weightedmat = create_useritem_matrix()
    df_item = compute_cosine_similarity_items(useritem)
    df_item_adj = compute_adjusted_cosine_similarity_items(useritem)
    df_user = compute_cosine_similarity_users(useritem)
    return useritem, weightedmat, df_item, df_item_adj, df_user

useritem, weightedmat, df_item, df_item_adj, df_user = initilaise_recommendation()
```

In [32]:
```
def recommendation_top5_with_scores(user_id):
    if user_id not in useritem.index:
        try:
            user_id = int(user_id)
        except ValueError:
            items = useritem.sum().nlargest(5)
            return list(zip(items.index.tolist(), items.values.tolist())), [], []
    if user_id not in useritem.index:
        items = useritem.sum().nlargest(5)
        return list(zip(items.index.tolist(), items.values.tolist())), [], []

    sims = df_user.loc[user_id].copy()
    sims[user_id] = 0.0
    scores_user = weightedmat.T.dot(sims)
    already = useritem.loc[user_id]
    scores_user.drop(already[already > 0].index, inplace=True)


    u_vec = weightedmat.loc[user_id]
    scores = u_vec.dot(df_item)
    purchased = u_vec[u_vec > 0].index
    scores = scores.drop(purchased, errors='ignore')

    u_vec2 = weightedmat.loc[user_id]
    scores_itemadj = u_vec2.dot(df_item_adj)
    purchased2 = u_vec2[u_vec2 > 0].index
    scores_itemadj = scores_itemadj.drop(purchased2, errors='ignore')


    top5item = scores.nlargest(5)
    top5user = scores_user.nlargest(5)
    top5itemadj = scores_itemadj.nlargest(5)

    top5item_pairs = list(zip(top5item.index.tolist(), top5item.values.tolist()))
    top5user_pairs = list(zip(top5user.index.tolist(), top5user.values.tolist()))
    top5itemadj_pairs = list(zip(top5itemadj.index.tolist(), top5itemadj.values.tolist())
```

```
    return top5item_pairs, top5user_pairs, top5itemadj_pairs
```

In [154…

```
def recommendation_top5(user_id, topn=5, t=0):
    if user_id not in useritem.index:
        try:
            user_id = int(user_id)
        except ValueError:
            items = useritem.sum().nlargest(topn)
            return list(zip(items.index.tolist(), items.values.tolist())), [], []
    if user_id not in useritem.index:
        items = useritem.sum().nlargest(topn)
        return list(zip(items.index.tolist(), items.values.tolist())), [], []

    sims = df_user.loc[user_id].copy()
    sims[user_id] = 0.0
    scores_user = weightedmat.T.dot(sims)
    already = useritem.loc[user_id]
    scores_user.drop(already[already > 0].index, inplace=True)


    u_vec = weightedmat.loc[user_id]
    scores = u_vec.dot(df_item)
    purchased = u_vec[u_vec > 0].index
    scores = scores.drop(purchased, errors='ignore')

    u_vec2 = weightedmat.loc[user_id]
    scores_itemadj = u_vec2.dot(df_item_adj)
    purchased2 = u_vec2[u_vec2 > 0].index
    scores_itemadj = scores_itemadj.drop(purchased2, errors='ignore')


    top5item = scores.nlargest(topn).index.tolist()
    top5user = scores_user.nlargest(topn).index.tolist()
    top5itemadj = scores_itemadj.nlargest(topn).index.tolist()
    if t:
        return top5user, top5item, top5itemadj
    else:
        return top5item, top5user, top5itemadj
```

Task 3: UI and Top level function code -Integration Code

In [157…

```
def gettop5recommendations(user_id, up, min_support=0.01, min_confidence=0.3, alpha=0.5):

    # ensure user exists
    if user_id not in useritem.index:
        freq_items, rules, _ = mine_patterns(train_data,min_support=min_support, min_conf
        if freq_items.empty:
            items = useritem.sum().nlargest(5)
            return list(zip(items.index, items.values))

        single = freq_items[freq_items['itemsets'].apply(len) == 1]
        single = single.sort_values('support', ascending=False).head(5)
        return [(next(iter(itemset)), row.support) for itemset, row in zip(single['itemse

    # load user purchase history
    user_hist = set(train_data.loc[train_data['User_id']==user_id, 'itemDescription'])

    if up:
        # Task 1: mine patterns for this user
        freq_items, rules, _ = mine_patterns(train_data, min_support=min_support, min_cor
        scored_patterns = score_patterns(freq_items, rules=rules, data=train_data, user_i
```

```
        top5_pats = scored_patterns.nlargest(5, 'score')
        pat_scores = {}
        for _, row in top5_pats.iterrows():
            for item in row['itemsets']:
                if item not in user_hist:       # don't re-recommend old buys
                # if the same item appears in multiple top patterns, keep the highest
                    pat_scores[item] = max(pat_scores.get(item, 0.0), row['score'])

        cf_recs, _, _ = recommendation_top5_with_scores(user_id)
        cf_scores  = {item: s for item, s in cf_recs if item not in user_hist}

        # Combine into hybrid scores
        hybrid_scores = {}
        for item in set(cf_scores) | set(pat_scores):
            hybrid_scores[item] = max(cf_scores.get(item, 0.0),alpha * pat_scores.get(ite
        # sort and return top 5
        return sorted(hybrid_scores.items(), key=lambda x: x[1], reverse=True)[:5]

    else:
        # Task 2 only: pure CF
        cf_recs, _, _ = recommendation_top5_with_scores(user_id)
        # filter out already purchased
        filtered = [(item, s) for item, s in cf_recs if item not in user_hist]
        return filtered[:5]
```

In [159…

```
while True:
    userin = input("Enter a User ID (Type 'exit' to quit): ")
    if userin.lower() == 'exit':
        break
    try:
        userin = int(userin)
    except:
        print("Please enter a Number")
        continue
    up = input("With Frequent Patterns (y/n): ").strip().lower() == 'y'
    # Getting the top 5 recommendations based on user's choice
    recommendations = gettop5recommendations(userin, up)
    if recommendations:
        if up:
            print("\nTop 5 recommendations for user with Frequent Patterns" + str(userin)
            print(f"{'Item':<30} {'Score':<10}")
            print("-" * 40)
            for item, score in recommendations:
                print(f"{item:<30} {score:.4f}")
        else:
            print("\nTop 5 recommendations for user without Frequent Patterns" + str(user
            print(f"{'Item':<30} {'Score':<10}")
            print("-" * 40)
            for item, score in recommendations:
                print(f"{item:<30} {score:.4f}")
    else:
        # If there was norecommendations were generated
        print("\n No recommendations could be generated. \n")
```

```
Top 5 recommendations for user with Frequent Patterns1991 :
Item                           Score
----------------------------------------
whole milk                     0.2587
rolls/buns                     0.2385
other vegetables               0.2328
pastry                         0.1803
root vegetables                0.1722
```

```
---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
Cell In[159], line 2
      1 while True:
----> 2     userin = input("Enter a User ID (Type 'exit' to quit): ")
      3     if userin.lower() == 'exit':
      4         break

File ~\anaconda3\Lib\site-packages\ipykernel\kernelbase.py:1262, in Kernel.raw_input(self,
prompt)
   1260     msg = "raw_input was called, but this frontend does not support input request
s."
   1261     raise StdinNotImplementedError(msg)
-> 1262 return self._input_request(
   1263     str(prompt),
   1264     self._parent_ident["shell"],
   1265     self.get_parent("shell"),
   1266     password=False,
   1267 )

File ~\anaconda3\Lib\site-packages\ipykernel\kernelbase.py:1305, in Kernel._input_request
(self, prompt, ident, parent, password)
   1302 except KeyboardInterrupt:
   1303     # re-raise KeyboardInterrupt, to truncate traceback
   1304     msg = "Interrupted by user"
-> 1305     raise KeyboardInterrupt(msg) from None
   1306 except Exception:
   1307     self.log.warning("Invalid Message:", exc_info=True)

KeyboardInterrupt: Interrupted by user
```

Testing and Evaluation

In [205…

```
min_support=0.0005
min_confidence=0.01
#Mining frequent patterns and association rules globally
freq_items, rules, _ = mine_patterns(train_data,min_support=min_support,min_confidence=mi
scored_patterns = score_patterns(freq_items,rules=rules,data=train_data,user_id=None,meth
```

In [ ]:

```
count=0
for _, row in scored_patterns.iterrows():
            if len(row['itemsets'])>=2 and row['score']>=0.002:
                count=count+1
                print(row)
print(count)
```

In [247…

```
def gettop5recommendations_evaluation(user_id, up, t=0, min_support=0.0005, min_confidenc
    # building the user history set
    user_hist = set(useritem.columns[useritem.loc[user_id] > 0])
    # print(user_hist)
    # 1) pure Collaborative Filtering(CF) branch
    if up == 0:
        top5item, top5user, top5itemadj = recommendation_top5(user_id,5,t)
        return top5item  # just the item-based CF list

    # 2) hybrid branch (pattern first, then CF)
    # 2a) mine + score patterns

    # 2b) take Top 100 patterns by the score column
    top100p = scored_patterns.nlargest(400, 'score')
    # print(top10p)
    # 2c) flatten those into a list of items.
    hybrid = []
```

```
    hybrid_final = []

    for _, row in top100p.iterrows():
            if len(row['itemsets'])>=2:
                fl=0
                for item in row['itemsets']:
                    if item in user_hist:
                         fl=1
                         for item2 in row['itemsets']:
                             if item2 not in hybrid and item2 not in user_hist and row['s
                                 hybrid.append(item2)
                    if fl==1:
                        break



    cf_items, _, _ = recommendation_top5(user_id, 30, t)
    for item in cf_items:
        # print(item)
        if item in hybrid:
            hybrid_final.append(item)
            # print(hybrid_final)
            if len(hybrid_final) == 5:
                return hybrid_final
    for item in cf_items:
        if item not in hybrid_final:
            hybrid_final.append(item)
            if len(hybrid_final) == 5:
                return hybrid_final
    return cf_items
    # print(hybrid)
```

In [249…

```
import time
def evaluate_cf():
    #creating a dictionary
    actual = dfte.groupby('user_id')['itemDescription'].apply(set).to_dict()
    # storing evaluation results for each user
    records = []
    start_time = time.time()
    users_processed = 0
    for uid, true_items in actual.items():
        if uid not in useritem.index:
            continue
        users_processed += 1
        # Getting top-5 recommendations for this user
        rec_item = gettop5recommendations_evaluation(uid,1)

        def rank_metrics(recs, true_set):
            hits = 0
            for item in recs:
                if item in true_set:
                    hits += 1
            precision = hits / 5
            if len(true_set) > 0:
                recall = hits / len(true_set)
            else:
                recall = np.nan
            if (precision + recall) > 0:
                f1 = 2 * precision * recall / (precision + recall)
            else:
                f1 = 0.0
                # Compute Mean Average Precision
            ap = 0.0
            hit_count = 0
            for k in range(1, 6):
```

```
                candidate = recs[k-1]
                if candidate in true_set:
                    hit_count += 1
                    ap += hit_count / k

            if len(true_set) > 0:
                denom = min(len(true_set), 5)
                ap = ap / denom
            else:
                ap = np.nan
            return precision, recall, f1, ap

        # Evaluating recommendations
        p_i, r_i, f_i, m_i = rank_metrics(rec_item, true_items)
        # p_u, r_u, f_u, m_u = rank_metrics(rec_user, true_items)

        # ratinging metrics on positives
        ps_i = weightedmat.loc[uid].dot(df_item)
        errs_i = [abs(ps_i[i] - 1) for i in true_items]
        mae_i = np.mean(errs_i) if errs_i else np.nan
        rmse_i = np.sqrt(np.mean([(ps_i[i] - 1)**2 for i in true_items])) if errs_i else


        records.append({'user_id': uid,'precision_item': p_i, 'recall_item': r_i, 'f1_ite
            # 'precision_user': p_u, 'recall_user': r_u, 'f1_user': f_u, 'map_user': m_u,
                    })
        total_time = time.time() - start_time
        avg_time_per_user = total_time / users_processed if users_processed else np.nan
        # Computing mean scores across users
    df_res = pd.DataFrame(records)
    summary = df_res.mean(numeric_only=True)
    print("=== Evaluation Summary ===")
    print("Average Time Per User = ", avg_time_per_user)
    print(f"Item-based  — Prec@5:{summary['precision_item']:.4f}, Rec@5:{summary['recall_
    # print(f"User-based  — Prec@5:{summary['precision_user']:.4f}, Rec@5:{summary['recal
    return df_res
# Loading the test dataset
dfte = pd.read_csv('Groceries data test.csv')
dfte['date'] = pd.to_datetime(dfte['Date'], dayfirst=True)
results = evaluate_cf()
```

```
=== Evaluation Summary ===
Average Time Per User =  0.05181645140610767
Item-based  — Prec@5:0.2084, Rec@5:0.1959, F1:0.1851, MAP:0.1648, MAE:0.9683, RMSE:0.9699
```

In [253…

```
def evaluate_cf2():
    #Building a dictionary of user_id
    actual = dfte.groupby('user_id')['itemDescription'].apply(set).to_dict()
    records = []
    start_time = time.time()
    users_processed = 0
    # Looping through each user and evaluate
    for uid, true_items in actual.items():
        if uid not in useritem.index:
            continue
        users_processed += 1
        #Generating top-5 recommendations using only Collaborative Filtering
        rec_item = gettop5recommendations_evaluation(uid,0)
# Defining a function to compute ranking metrics
        def rank_metrics(recs, true_set):
            hits = 0
            for item in recs:
                if item in true_set:
                    hits += 1
            precision = hits / 5
            if len(true_set) > 0:
```

```
                recall = hits / len(true_set)
            else:
                recall = np.nan
            if (precision + recall) > 0:
                f1 = 2 * precision * recall / (precision + recall)
            else:
                f1 = 0.0
                # Computing Mean Average Precision
            ap = 0.0
            hit_count = 0
            for k in range(1, 6):
                candidate = recs[k-1]
                if candidate in true_set:
                    hit_count += 1
                    ap += hit_count / k

            if len(true_set) > 0:
                denom = min(len(true_set), 5)
                ap = ap / denom
            else:
                ap = np.nan
            return precision, recall, f1, ap

# Calculating ranking metrics for this user
        p_i, r_i, f_i, m_i = rank_metrics(rec_item, true_items)

        # Calculating rating prediction errors
        ps_i = weightedmat.loc[uid].dot(df_item)
        errs_i = [abs(ps_i[i] - 1) for i in true_items]
        mae_i = np.mean(errs_i) if errs_i else np.nan
        rmse_i = np.sqrt(np.mean([(ps_i[i] - 1)**2 for i in true_items])) if errs_i else


        records.append({'user_id': uid,'precision_item': p_i, 'recall_item': r_i, 'f1_ite
            # 'precision_user': p_u, 'recall_user': r_u, 'f1_user': f_u, 'map_user': m_u,
                        })
        total_time = time.time() - start_time
        avg_time_per_user = total_time / users_processed if users_processed else np.nan

    df_res = pd.DataFrame(records)
    summary = df_res.mean(numeric_only=True)
    print("=== Evaluation Summary ===")
    print("Average Time Per User = ", avg_time_per_user)
    print(f"Only Collaborative Filtering  — Prec@5:{summary['precision_item']:.4f}, Rec@5
    # print(f"User-based  — Prec@5:{summary['precision_user']:.4f}, Rec@5:{summary['recal
    return df_res

dfte = pd.read_csv('Groceries data test.csv')
dfte['date'] = pd.to_datetime(dfte['Date'], dayfirst=True)
results2 = evaluate_cf2()
```

```
=== Evaluation Summary ===
Average Time Per User =  0.010267252797196287
Only Collaborative Filtering  — Prec@5:0.2067, Rec@5:0.1944, F1:0.1836, MAP:0.1639, MAE:0.
9683, RMSE:0.9699
```

In [169…

```
def evaluate_cf3():
    actual = dfte.groupby('user_id')['itemDescription'].apply(set).to_dict()
    records = []
    for uid, true_items in actual.items():
        if uid not in useritem.index:
            continue
        rec_item = gettop5recommendations_evaluation(uid,0,1)

        def rank_metrics(recs, true_set):
            hits = 0
```

```
            for item in recs:
                if item in true_set:
                    hits += 1
            precision = hits / 5
            if len(true_set) > 0:
                recall = hits / len(true_set)
            else:
                recall = np.nan
            if (precision + recall) > 0:
                f1 = 2 * precision * recall / (precision + recall)
            else:
                f1 = 0.0
            ap = 0.0
            hit_count = 0
            for k in range(1, 6):
                candidate = recs[k-1]
                if candidate in true_set:
                    hit_count += 1
                    ap += hit_count / k

            if len(true_set) > 0:
                denom = min(len(true_set), 5)
                ap = ap / denom
            else:
                ap = np.nan
            return precision, recall, f1, ap

        p_i, r_i, f_i, m_i = rank_metrics(rec_item, true_items)
        # p_u, r_u, f_u, m_u = rank_metrics(rec_user, true_items)

        # rating metrics on positives
        # ps_i = weightedmat.loc[uid].dot(df_item)
        # errs_i = [abs(ps_i[i] - 1) for i in true_items]
        # mae_i = np.mean(errs_i) if errs_i else np.nan
        # rmse_i = np.sqrt(np.mean([(ps_i[i] - 1)**2 for i in true_items])) if errs_i els

        # sims = df_user.loc[uid].copy(); sims[uid] = 0
        # ps_u = weightedmat.T.dot(sims)
        # errs_u = [abs(ps_u[i] - 1) for i in true_items]
        # mae_u = np.mean(errs_u) if errs_u else np.nan
        # rmse_u = np.sqrt(np.mean([(ps_u[i] - 1)**2 for i in true_items])) if errs_u els

        records.append({'user_id': uid,'precision_item': p_i, 'recall_item': r_i, 'f1_ite
            # 'precision_user': p_u, 'recall_user': r_u, 'f1_user': f_u, 'map_user': m_u,
                        })

    df_res = pd.DataFrame(records)
    summary = df_res.mean(numeric_only=True)
    print("=== Evaluation Summary ===")
    print(f"Only Collaborative Filtering (User Based) — Prec@5:{summary['precision_item']
    # print(f"User-based  — Prec@5:{summary['precision_user']:.4f}, Rec@5:{summary['recal
    return df_res

dfte = pd.read_csv('Groceries data test.csv')
dfte['date'] = pd.to_datetime(dfte['Date'], dayfirst=True)
results3 = evaluate_cf3()
```

```
=== Evaluation Summary ===
Only Collaborative Filtering (User Based) — Prec@5:0.2214, Rec@5:0.2072, F1:0.1966, MAP:0.
1722
```

In [171…

```
def evaluate_cf4():
    actual = dfte.groupby('user_id')['itemDescription'].apply(set).to_dict()
    records = []
    for uid, true_items in actual.items():
        if uid not in useritem.index:
```

```
            continue
        rec_item = gettop5recommendations_evaluation(uid,1,1)

        def rank_metrics(recs, true_set):
            hits = 0
            for item in recs:
                if item in true_set:
                    hits += 1
            precision = hits / 5
            if len(true_set) > 0:
                recall = hits / len(true_set)
            else:
                recall = np.nan
            if (precision + recall) > 0:
                f1 = 2 * precision * recall / (precision + recall)
            else:
                f1 = 0.0
            ap = 0.0
            hit_count = 0
            for k in range(1, 6):
                candidate = recs[k-1]
                if candidate in true_set:
                    hit_count += 1
                    ap += hit_count / k

            if len(true_set) > 0:
                denom = min(len(true_set), 5)
                ap = ap / denom
            else:
                ap = np.nan
            return precision, recall, f1, ap

        p_i, r_i, f_i, m_i = rank_metrics(rec_item, true_items)
        # p_u, r_u, f_u, m_u = rank_metrics(rec_user, true_items)

        # rating metrics on positives
        # ps_i = weightedmat.loc[uid].dot(df_item)
        # errs_i = [abs(ps_i[i] - 1) for i in true_items]
        # mae_i = np.mean(errs_i) if errs_i else np.nan
        # rmse_i = np.sqrt(np.mean([(ps_i[i] - 1)**2 for i in true_items])) if errs_i els

        # sims = df_user.loc[uid].copy(); sims[uid] = 0
        # ps_u = weightedmat.T.dot(sims)
        # errs_u = [abs(ps_u[i] - 1) for i in true_items]
        # mae_u = np.mean(errs_u) if errs_u else np.nan
        # rmse_u = np.sqrt(np.mean([(ps_u[i] - 1)**2 for i in true_items])) if errs_u els

        records.append({'user_id': uid,'precision_item': p_i, 'recall_item': r_i, 'f1_ite
            # 'precision_user': p_u, 'recall_user': r_u, 'f1_user': f_u, 'map_user': m_u,
                        })

    df_res = pd.DataFrame(records)
    summary = df_res.mean(numeric_only=True)
    print("=== Evaluation Summary ===")
    print(f"Hybrid Approach (User Based) — Prec@5:{summary['precision_item']:.4f}, Rec@5:
    # print(f"User-based  — Prec@5:{summary['precision_user']:.4f}, Rec@5:{summary['recal
    return df_res

dfte = pd.read_csv('Groceries data test.csv')
dfte['date'] = pd.to_datetime(dfte['Date'], dayfirst=True)
results4 = evaluate_cf4()
```

```
=== Evaluation Summary ===
Hybrid Approach (User Based) — Prec@5:0.2215, Rec@5:0.2076, F1:0.1968, MAP:0.1725
```

In [173…

```
min_support=0.0005
min_confidence=0.01
freq_items, rules, _ = mine_patterns(
        train_data,
        min_support=min_support,
        min_confidence=min_confidence,
        user_id=None
    )
scored_patterns2 = score_patterns(
        freq_items,
        rules=rules,
        data=train_data,
        user_id=None,
        method='confidence'
    )
```

In [ ]:

```
scored_patterns2.head(50)
```

In [189…

```
def gettop5recommendations_evaluation(user_id, up, t=0, min_support=0.0005, min_confidenc
    # build user history set
    user_hist = set(useritem.columns[useritem.loc[user_id] > 0])
    # print(user_hist)
    # 1) pure CF branch
    if up == 0:
        top5item, top5user, top5itemadj = recommendation_top5(user_id,5,t)
        return top5item  # just the item-based CF list

    # 2) hybrid branch (pattern first, then CF)
    # 2a) mine + score patterns

    # 2b) take Top 100 patterns by the core column
    top100p = scored_patterns2.nlargest(50, 'score')
    # print(top10p)
    # 2c) flatten those into a list of items (drop already-bought)
    hybrid = []
    hybrid_final = []

    for _, row in top100p.iterrows():
            if len(row['itemsets'])>=2:
                fl=0
                for item in row['itemsets']:
                    if item in user_hist:
                         fl=1
                         for item2 in row['itemsets']:
                             if item2 not in hybrid and item2 not in user_hist:
                                 hybrid.append(item2)
                    if fl==1:
                        break


    cf_items, _, _ = recommendation_top5(user_id, 30, t)
    for item in cf_items:
        # print(item)
        if item in hybrid:
            hybrid_final.append(item)
            # print(hybrid_final)
            if len(hybrid_final) == 5:
                return hybrid_final
    for item in cf_items:
        if item not in hybrid_final:
            hybrid_final.append(item)
            if len(hybrid_final) == 5:
                return hybrid_final
```

```
    return cf_items
    # print(hybrid)
```

In [191…

```
def evaluate_cf5():
    # Defining the function `evaluate_cf5`
    #for evaluating collaborative filtering-based recommendations.
    actual = dfte.groupby('user_id')['itemDescription'].apply(set).to_dict()
    records = []
    for uid, true_items in actual.items():
        if uid not in useritem.index:
            continue
        rec_item = gettop5recommendations_evaluation2(uid,1)
# Getting the top 5 recommended items for the user
        def rank_metrics(recs, true_set):
            hits = 0
            for item in recs:
                # Looping through the recommended items
                if item in true_set:
                    hits += 1
            precision = hits / 5
            if len(true_set) > 0:
                recall = hits / len(true_set)
            else:
                recall = np.nan
            if (precision + recall) > 0:
                f1 = 2 * precision * recall / (precision + recall)
            else:
                f1 = 0.0
            ap = 0.0
            hit_count = 0
            for k in range(1, 6):
                # Looping through the top 5 recommended items to compute the average prec
                candidate = recs[k-1]
                if candidate in true_set:
                    hit_count += 1
                    ap += hit_count / k

            if len(true_set) > 0:
                denom = min(len(true_set), 5)
                ap = ap / denom
            else:
                ap = np.nan
            return precision, recall, f1, ap

        p_i, r_i, f_i, m_i = rank_metrics(rec_item, true_items)

        # rating the metrics on positives
        ps_i = weightedmat.loc[uid].dot(df_item)
        errs_i = [abs(ps_i[i] - 1) for i in true_items]
        mae_i = np.mean(errs_i) if errs_i else np.nan
        rmse_i = np.sqrt(np.mean([(ps_i[i] - 1)**2 for i in true_items])) if errs_i else

        records.append({'user_id': uid,'precision_item': p_i, 'recall_item': r_i, 'f1_ite
            # 'precision_user': p_u, 'recall_user': r_u, 'f1_user': f_u, 'map_user': m_u,
                    })

    df_res = pd.DataFrame(records)
    summary = df_res.mean(numeric_only=True)
    print("=== Evaluation Summary ===")
    print(f"Hybrid Approach(Scoring on Confidence) — Prec@5:{summary['precision_item']:.4
    # print(f"User-based  — Prec@5:{summary['precision_user']:.4f}, Rec@5:{summary['recal
    # MAP:{summary['map_user']:.4f}, MAE:{summary['mae_user']:.4f}, RMSE:{summary['rmse_u
    return df_res
# Loading the 'Groceries data test.csv' file into a DataFrame
dfte = pd.read_csv('Groceries data test.csv')
```

```
dfte['date'] = pd.to_datetime(dfte['Date'], dayfirst=True)
results5 = evaluate_cf5()
```

```
=== Evaluation Summary ===
Hybrid Approach(Scoring on Confidence) — Prec@5:0.1833, Rec@5:0.1696, F1:0.1618, MAP:0.133
2, MAE:0.9683, RMSE:0.9699
```

In [197…

```
results.head()
```

Out[197…

	user_id	precision_item	recall_item	f1_item	map_item	mae_item	rmse_item
0	1000	0.0	0.0	0.0	0.0	0.999988	0.999988
1	1001	0.0	0.0	0.0	0.0	0.644973	0.692928
2	1002	0.0	0.0	0.0	0.0	0.999828	0.999828
3	1003	0.0	0.0	0.0	0.0	0.998411	0.998412
4	1004	0.0	0.0	0.0	0.0	0.990822	0.990833

In [211…

```
metrics = ['precision_item','recall_item','f1_item','map_item']

hybrid = results[metrics].mean()
cf = results2[metrics].mean()
# summary_user     = df_user[metrics].mean()

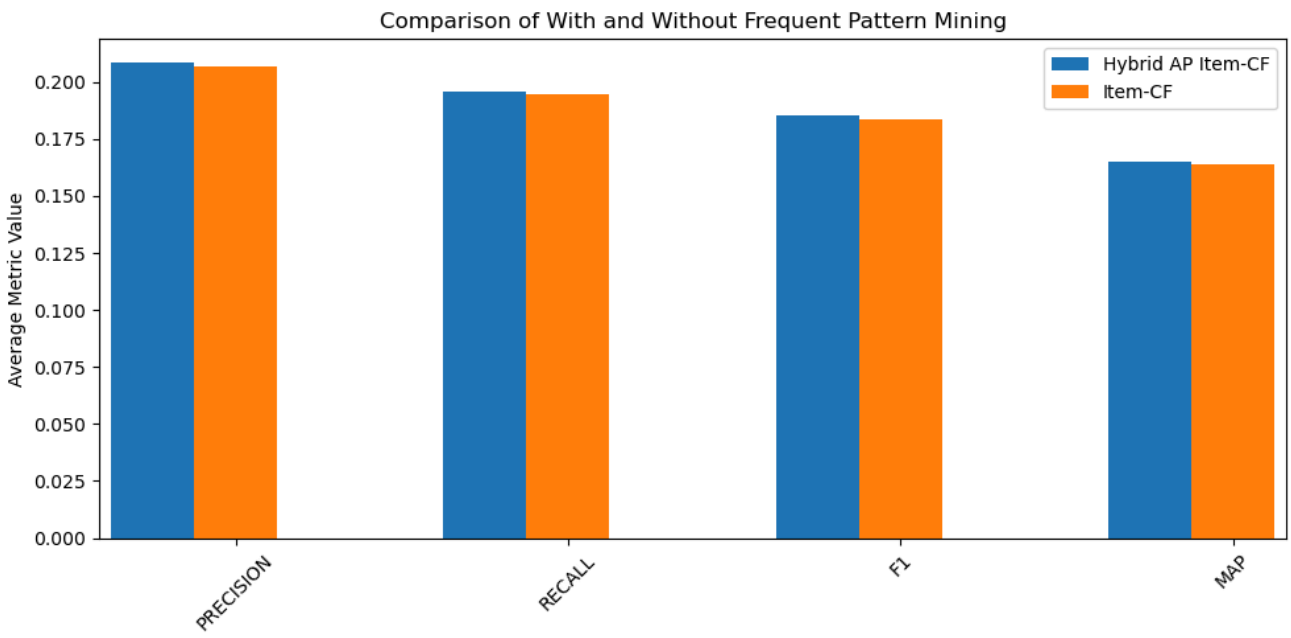
# Combining into one DataFrame for ease of plotting:
summary = pd.DataFrame({
    'Hybrid AP Item-CF': hybrid,
    'Item-CF': cf,
    # 'User-CF':          summary_user
})

x     = np.arange(len(metrics))
# the label locations
width = 0.25
# the width of the bars

fig, ax = plt.subplots(figsize=(10,5))
ax.bar(x - width, summary['Hybrid AP Item-CF'], width, label='Hybrid AP Item-CF')
ax.bar(x,       summary['Item-CF'], width, label='Item-CF')


ax.set_xticks(x)
ax.set_xticklabels([m.replace('_item','').upper() for m in metrics], rotation=45)
ax.set_ylabel('Average Metric Value')
ax.set_title('Comparison of With and Without Frequent Pattern Mining')
ax.legend()
ax.margins(x=0.01)

plt.tight_layout()
plt.show()
```



In [217…

```
metrics = ['precision_item','recall_item','f1_item','map_item']

hybrid = results4[metrics].mean()
cf = results3[metrics].mean()
# summary_user     = df_user[metrics].mean()

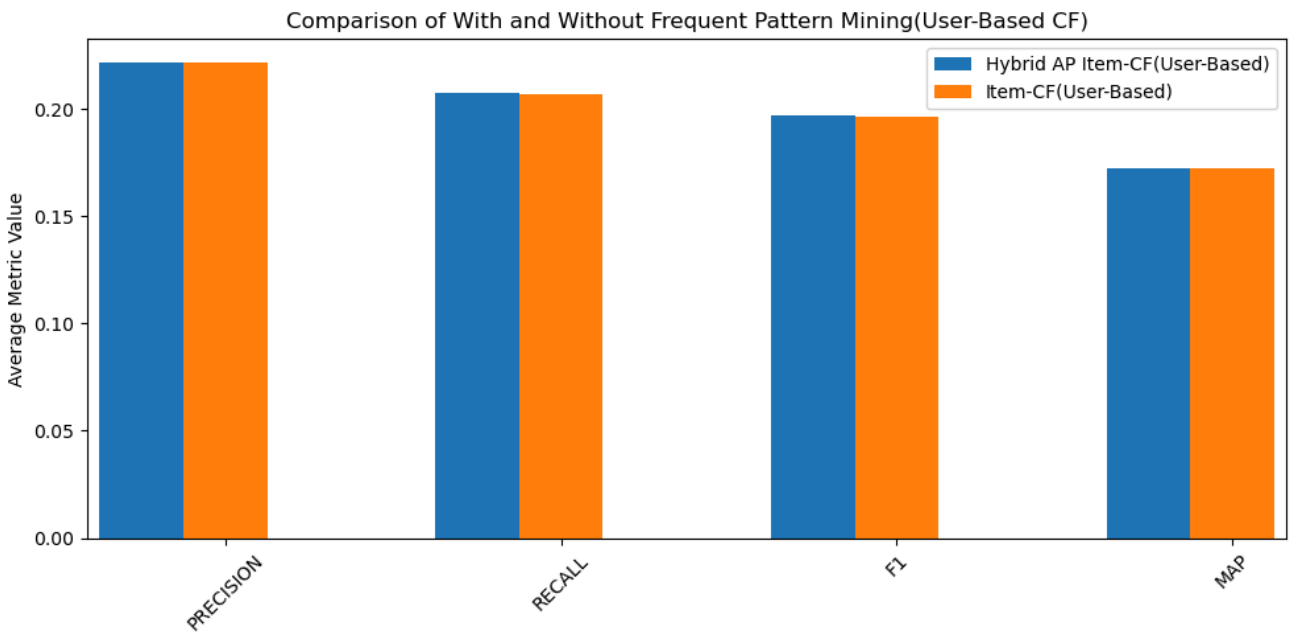
# Combining into one DataFrame for ease of plotting:
summary = pd.DataFrame({
    'Hybrid AP Item-CF': hybrid,
    'Item-CF': cf,
    # 'User-CF':          summary_user
})

x     = np.arange(len(metrics))
# the label locations
width = 0.25
# the width of the bars

fig, ax = plt.subplots(figsize=(10,5))
ax.bar(x - width, summary['Hybrid AP Item-CF'], width, label='Hybrid AP Item-CF(User-Base
ax.bar(x,       summary['Item-CF'], width, label='Item-CF(User-Based)')


ax.set_xticks(x)
ax.set_xticklabels([m.replace('_item','').upper() for m in metrics], rotation=45)
ax.set_ylabel('Average Metric Value')
ax.set_title('Comparison of With and Without Frequent Pattern Mining(User-Based CF)')
ax.legend()
ax.margins(x=0.01)

plt.tight_layout()
plt.show()
```


Comparison of With and Without Frequent Pattern Mining(User-Based CF)
Hybrid AP Item-CF(User-Based)
Item-CF(User-Based)
Average Metric Value
0.00
0.05
0.10
0.15
0.20
PRECISION
RECALL
F1
MAP

In [ ]: